[University name]
Faculty of [Faculty] · Department of [Department]
Course: Software Design Patterns

FINAL PROJECT REPORT

ByteForge

Custom PC Configurator & Computer Store
built with eleven cooperating design patterns

Team members	Patterns implemented
Student 1: [Full name], [group]	Singleton, Factory, Adapter, State
Student 2: [Full name], [group]	Builder (+ Director), Decorator, Command
Student 3: [Full name], [group]	Strategy, Observer, Facade, Chain of Responsibility
Supervisor	[Senior Lecturer name]

[City], 2026

Contents

1. Introduction
2. System overview and architecture
3. Design patterns (body)
    3.1 Singleton — Inventory
    3.2 Factory — ComponentFactory
    3.3 Adapter — supplier feeds
    3.4 Builder — PCBuilder and BuildDirector
    3.5 Decorator — service add-ons
    3.6 Strategy — pricing policies
    3.7 Observer — order and stock events
    3.8 Facade — StoreFacade
    3.9 Command — undo / redo
    3.10 State — order life cycle
    3.11 Chain of Responsibility — compatibility rules
4. How the patterns cooperate
5. Application screenshots
6. Testing and results
7. Conclusion
8. Further work

Appendix A. Project structure and how to run

1. Introduction

1.1 Problem statement

Buying a custom-built computer is error-prone. A processor only fits a motherboard with the same socket, memory must match the board's DDR generation, a long graphics card may not fit into a compact case, and an under-sized power supply makes the whole system unstable. Small computer shops usually check this by hand, take their part lists from several wholesalers in different formats, and price the same machine differently for students, companies and seasonal campaigns.

ByteForge is a desktop application for such a shop. A customer chooses parts (or starts from a ready-made preset), the system instantly checks hardware compatibility and power consumption, lets the customer add services such as an operating-system installation or an extended warranty, calculates the price with the correct pricing policy, and then tracks the order from payment through assembly, testing and delivery.

1.2 Objectives

- Build one cohesive Java application in which **at least six** design patterns from the course work together. We implemented **eleven**: Singleton, Factory, Adapter, Builder, Decorator, Facade, Strategy, Observer, Command, State and Chain of Responsibility.
- Each team member implements at least two patterns (see the cover page for the split).
- Show every pattern running in a graphical user interface and in a console demonstration, and verify it with automated tests.
- Document the design with UML class and sequence diagrams.

1.3 Technologies

Item	Choice
Language	Java 11+ (developed and tested with JDK 17, Temurin)
User interface	Java Swing with the Nimbus look and feel
Build	Plain `javac` (no external libraries); `run.bat` script; ready-made IntelliJ IDEA project with run configurations
Testing	Own dependency-free test runner (`run.bat --test`), 45 automated tests
Diagrams	UML class and sequence diagrams (Mermaid notation)
Size	43 source files, about 4,000 lines of code, 12 packages

1.4 Main features

- Catalog of 23 real-world parts imported from two simulated suppliers with incompatible data formats (CSV text and JSON-like REST data).

- Three presets (Office Starter, Gaming Pro, Creator Workstation) and free part selection.
- Live compatibility check by a chain of nine rules: required parts, CPU socket, RAM type, RAM slots (2 on mini-ITX boards), motherboard form factor vs. case, GPU length vs. case clearance, drive bays, missing integrated graphics, PSU wattage with 25 % headroom.
- Stackable services: Windows installation, ARGB lighting kit, overclocking, extended warranty (priced as a percentage of everything it wraps).
- Four pricing policies: regular, student, corporate bulk tiers and a seasonal sale.
- Unlimited **undo / redo** (Ctrl+Z / Ctrl+Y) of every configuration change; loading a preset is undone as one step.
- Checkout with a realistic two-step payment: the card is *authorized* first, stock is reserved, then the payment is *captured*. A declined card never touches the inventory; if stock is short, the card hold is released.
- Order tracking with notifications to the customer, the workshop and a sales dashboard; low-stock alerts to purchasing.
- Order cancellation with refund and automatic return of parts to stock, allowed only before assembly starts.
- Inventory screen with search, category and low-stock filters, and restocking.

2. System overview and architecture

The code is organised by responsibility, and each package is home to one pattern. The user interface (`ui`) and the console demo talk *only* to `StoreFacade`; the facade coordinates every other subsystem.

Package	Responsibility	Pattern
`model`	Hardware parts: `Component`, `Cpu`, `Motherboard`, `Ram`, `Gpu`, `Storage`, `PowerSupply`, `PcCase`	—
`adapter`	Import parts from suppliers with foreign formats	Adapter
`factory`	Create the right `Component` subclass from a neutral record	Factory
`inventory`	The single stock ledger of the shop	Singleton (+ Observer subject)
`builder`	Assemble and validate a `Computer`; presets	Builder + Director
`decorator`	Add optional paid services to a computer	Decorator
`strategy`	Interchangeable pricing policies	Strategy
`observer`	Orders, order status and event listeners	Observer
`facade`	One simple API for the UI: catalog, quote, checkout, tracking	Facade
`command`	Undoable configuration changes and their history	Command
`state`	Behaviour of each step of the order life cycle	State
`rules`	Hardware compatibility rules linked into a chain	Chain of Responsibility
`ui`	Swing window and the screenshot tour	—
`test/`	45 automated tests for all patterns	—

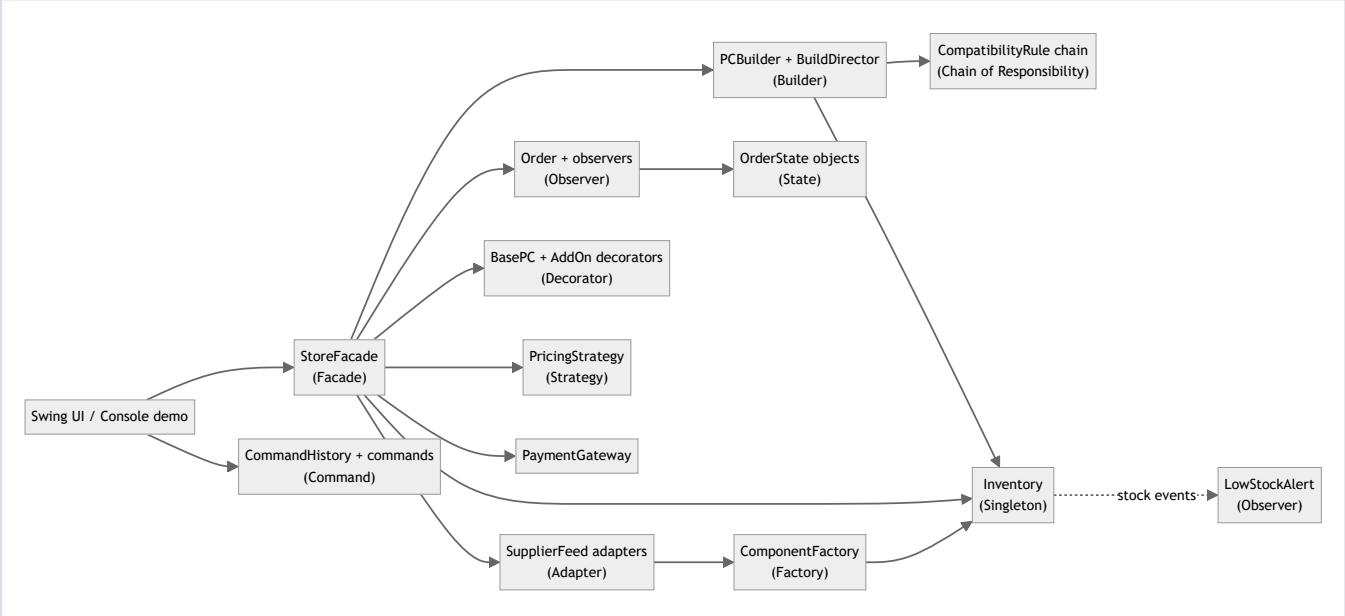


Figure 2.1 – High-level architecture: which pattern lives where and who calls whom.

2.1 Overall class diagram

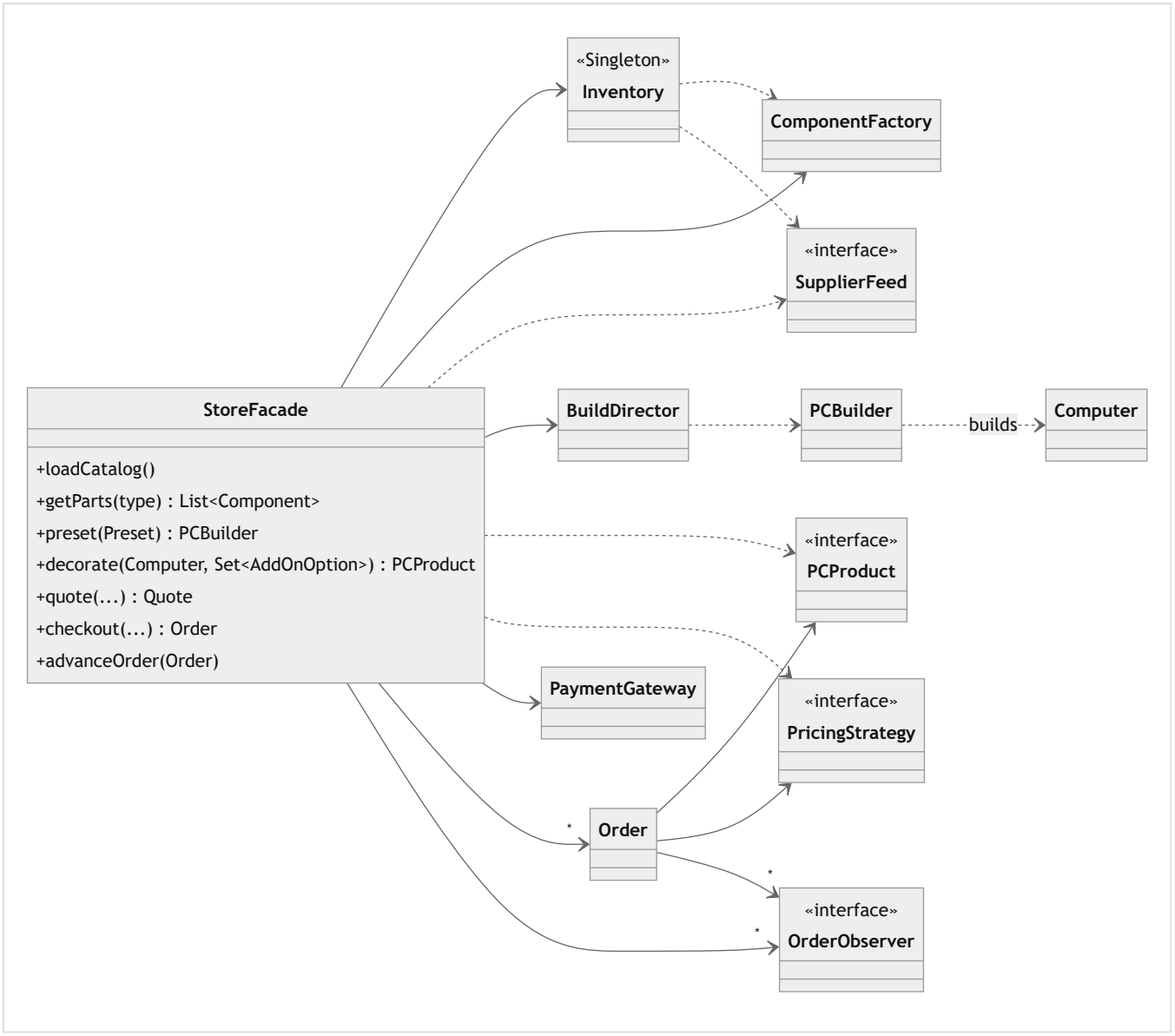


Figure 2.2 – Main classes and their relations (details per pattern in Chapter 3).

3. Design patterns

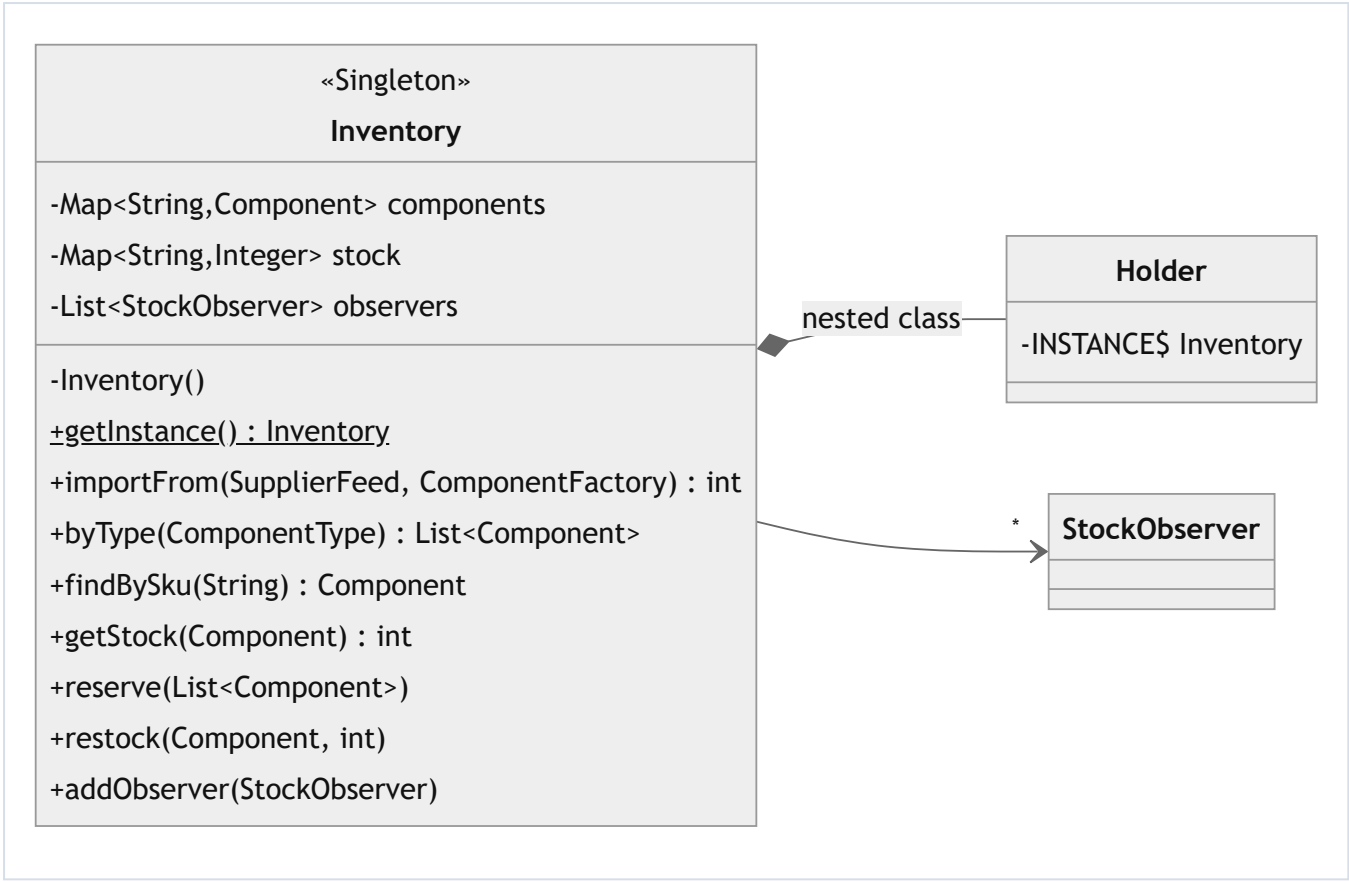
For every pattern we describe the problem it solves in ByteForge, the participating classes, a UML class diagram and the key part of the source code. Full source code is in the attached ZIP archive.

#	Pattern	Category	Main classes	Author
1	Singleton	Creational	`Inventory`	Student 1
2	Factory	Creational	`ComponentFactory`, `ComponentRecord`	Student 1
3	Adapter	Structural	`SupplierFeed`, `NovaTechCsvAdapter`, `PartHubApiAdapter`	Student 1
4	Builder	Creational	`PCBuilder`, `Computer`, `BuildDirector`	Student 2
5	Decorator	Structural	`PCProduct`, `BasePC`, `AddOnDecorator`, `AddOns.*`	Student 2
6	Strategy	Behavioral	`PricingStrategy`, `PricingStrategies.*`	Student 3
7	Observer	Behavioral	`Order`, `OrderObserver`, `StockObserver`, `Listeners.*`	Student 3
8	Facade	Structural	`StoreFacade`	Student 3
9	Command	Behavioral	`Command`, `ConfigCommands.*`, `CommandHistory`	Student 2
10	State	Behavioral	`OrderState`, `OrderStates.*`, `Order` (context)	Student 1
11	Chain of Responsibility	Behavioral	`CompatibilityRule`, `CompatibilityRules.*`	Student 3

3.1 Singleton — `Inventory` Creational

Problem. The shop has exactly one stock of physical parts. The configurator, the checkout, the inventory screen and the low-stock alert must all read and change the *same* quantities; two separate copies would let two customers buy the last RTX 4090.

Solution. `Inventory` has a private constructor and is reachable only through `getInstance()`. We use the *initialization-on-demand holder* idiom: the instance is created lazily the first time `Holder` is touched, and the JVM's class-loading rules make this thread-safe without `synchronized` on every call. All state-changing methods are `synchronized`, and `reserve()` is atomic: either every unit of an order is taken or nothing changes.




```
public final class Inventory {
    private Inventory() { }                       // nobody else can call "new"

    private static final class Holder {           // loaded only on first use
        private static final Inventory INSTANCE = new Inventory();
    }
    public static Inventory getInstance() { return Holder.INSTANCE; }

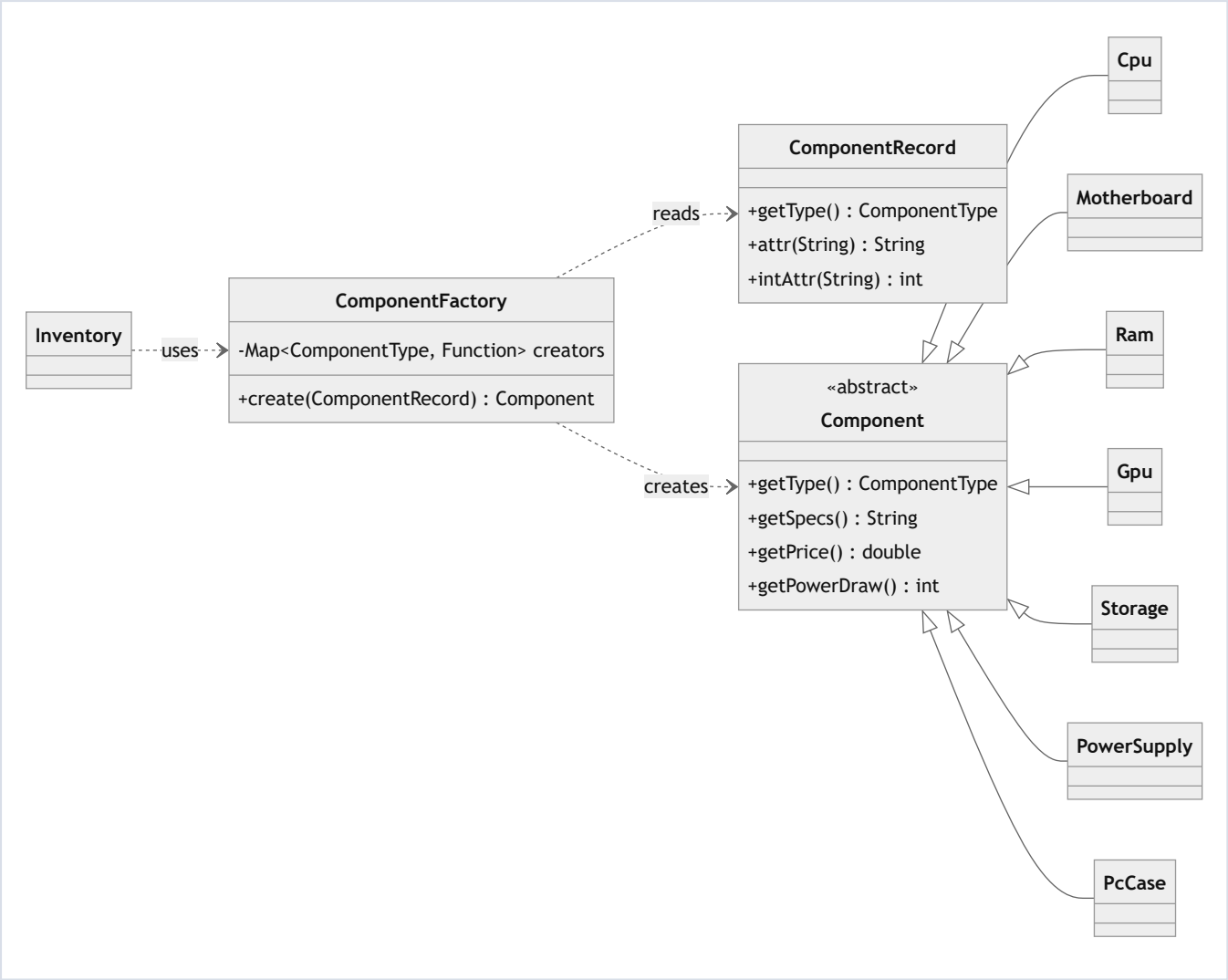
    /** Either every unit is taken from stock or, if anything is short, nothing changes. */
    public synchronized void reserve(List<Component> parts) {
        Map<Component, Integer> needed = new LinkedHashMap<>();
        for (Component c : parts) needed.merge(c, 1, Integer::sum);
        for (Map.Entry<Component, Integer> e : needed.entrySet())
            if (getStock(e.getKey()) < e.getValue())
                throw new IllegalStateException("Not enough stock for " + e.getKey().getName());
        for (Map.Entry<Component, Integer> e : needed.entrySet())
            changeStock(e.getKey(), -e.getValue());   // notifies StockObservers
    }
}
```

3.2 Factory — `ComponentFactory` Creational

Problem. Supplier data arrives as generic records ("type = GPU, lengthMm = 267 ..."). Seven concrete part classes exist, each with its own constructor and attributes. If the inventory loader called these constructors directly it would contain a large `if/else` over types and would have to change for every new part type.

Solution. `ComponentFactory.create(ComponentRecord)` is the only place that knows which concrete class matches a `ComponentType` and how to read its attributes. Clients receive the abstract `Component`.

Creators are registered in an `EnumMap`, so a new part type (e.g. a CPU cooler) is added by registering one more creator function.




```
public final class ComponentFactory {
    private final Map<ComponentType, Function<ComponentRecord, Component>> creators =
            new EnumMap<>(ComponentType.class);

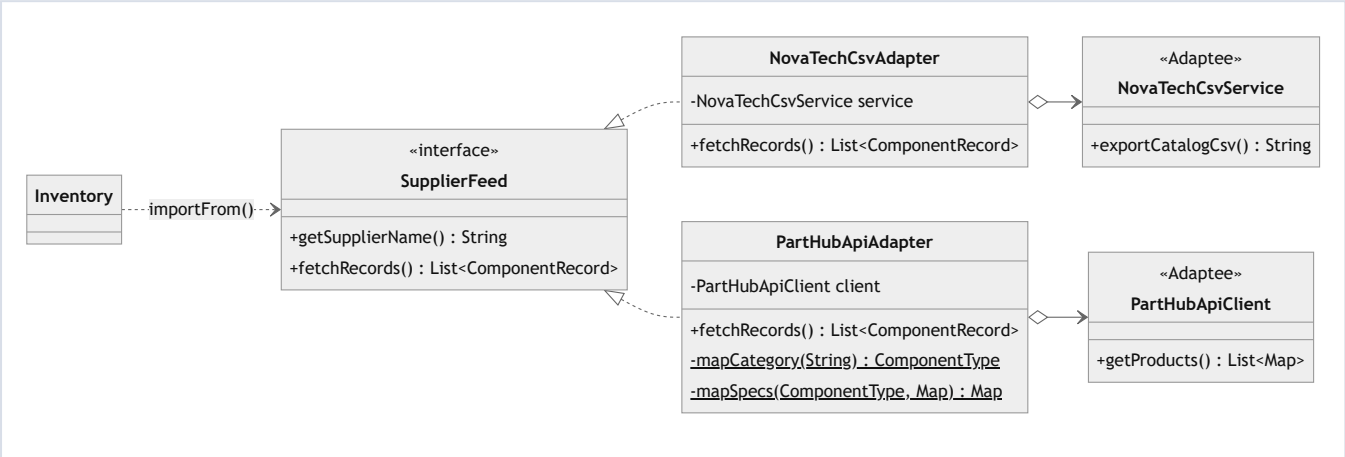
    public ComponentFactory() {
        creators.put(ComponentType.CPU, r -> new Parts.Cpu(
                r.getSku(), r.getBrand(), r.getModel(), r.getPrice(), r.intAttr("tdp"),
                r.attr("socket"), r.intAttr("cores"), r.doubleAttr("ghz")));
        creators.put(ComponentType.GPU, r -> new Parts.Gpu(
                r.getSku(), r.getBrand(), r.getModel(), r.getPrice(), r.intAttr("tdp"),
                r.intAttr("vramGb"), r.intAttr("lengthMm")));
        // ... MOTHERBOARD, RAM, STORAGE, PSU, CASE
    }

    public Component create(ComponentRecord record) {
        Function<ComponentRecord, Component> creator = creators.get(record.getType());
        if (creator == null) throw new IllegalArgumentException("No creator for " + record.getType());
        return creator.apply(record);
    }
}
```

3.3 Adapter — supplier feeds Structural

Problem. The shop buys from two suppliers whose code we cannot change. *NovaTech Distribution* (a legacy wholesaler) only exports semicolon-separated text. *PartHub* (a modern REST API) returns JSON-like maps, uses its own category names ("graphics", "chassis"), prices in cents and different spec keys ("boardPower" instead of "tdp").

Solution. The store defines the target interface `SupplierFeed`. Two *object adapters* wrap the incompatible services and translate their data into neutral `ComponentRecord`s. `Inventory.importFrom()` works with any `SupplierFeed`, so a third supplier needs only a new adapter.



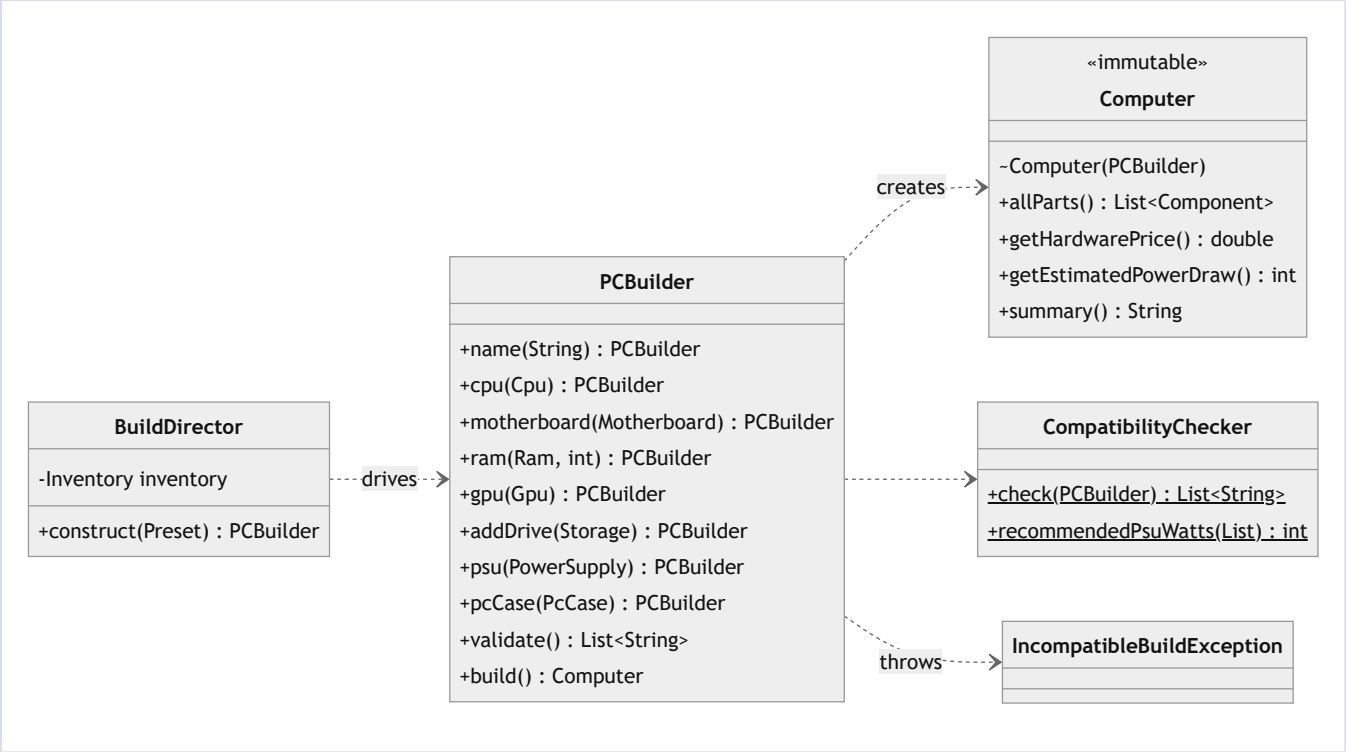

```
public class PartHubApiAdapter implements SupplierFeed {
    private final PartHubApiClient client;                  // the adaptee
    public PartHubApiAdapter(PartHubApiClient client) { this.client = client; }

    @Override public List<ComponentRecord> fetchRecords() {
        List<ComponentRecord> records = new ArrayList<>();
        for (Map<String, Object> p : client.getProducts()) {
            ComponentType type = mapCategory((String) p.get("category")); // "graphics" -> GPU
            records.add(new ComponentRecord((String) p.get("id"), type,
                    (String) p.get("manufacturer"), (String) p.get("name"),
                    ((Integer) p.get("priceCents")) / 100.0,              // cents -> dollars
                    (Integer) p.get("qty"),
                    mapSpecs(type, (Map<String, String>) p.get("specs"))));   // rename keys
        }
        return records;
    }
}
```

3.4 Builder — `PCBuilder`, `Computer`, `BuildDirector` Creational

Problem. A computer has eight kinds of parts; some are optional (graphics card), some repeat (RAM modules, drives). A constructor with ten parameters would be unreadable, and an object built field-by-field with setters could exist in an invalid state (e.g. Intel CPU on an AMD board).

Solution. `PCBuilder` collects parts through a fluent API. `build()` runs the `CompatibilityChecker` and only then creates an *immutable* `Computer` (its constructor is package-private, so the builder is the only way to obtain one). The GUI calls `validate()` after every change to show problems live. `BuildDirector` plays the Director role: it knows the recipes of the shop's three presets and returns a pre-filled builder that the customer can still modify.




```
// Director: a preset recipe
private PCBuilder gaming() {
    return new PCBuilder()
            .name("Gaming Pro")
            .cpu(inventory.findBySku("NT-CPU-02"))          // Ryzen 7 7800X3D
            .motherboard(inventory.findBySku("NT-MB-01"))   // B650, AM5, DDR5
            .ram(inventory.findBySku("PH-1001"), 2)         // 2 x 16 GB DDR5
            .gpu(inventory.findBySku("PH-2002"))            // Radeon RX 7800 XT
            .addDrive(inventory.findBySku("PH-3001"))
            .psu(inventory.findBySku("NT-PSU-02"))          // 850 W
            .pcCase(inventory.findBySku("PH-4001"));
}

// Builder: the product exists only if it is valid
public Computer build() {
    List<String> issues = validate();
    if (!issues.isEmpty()) throw new IncompatibleBuildException(issues);
    return new Computer(this);
}

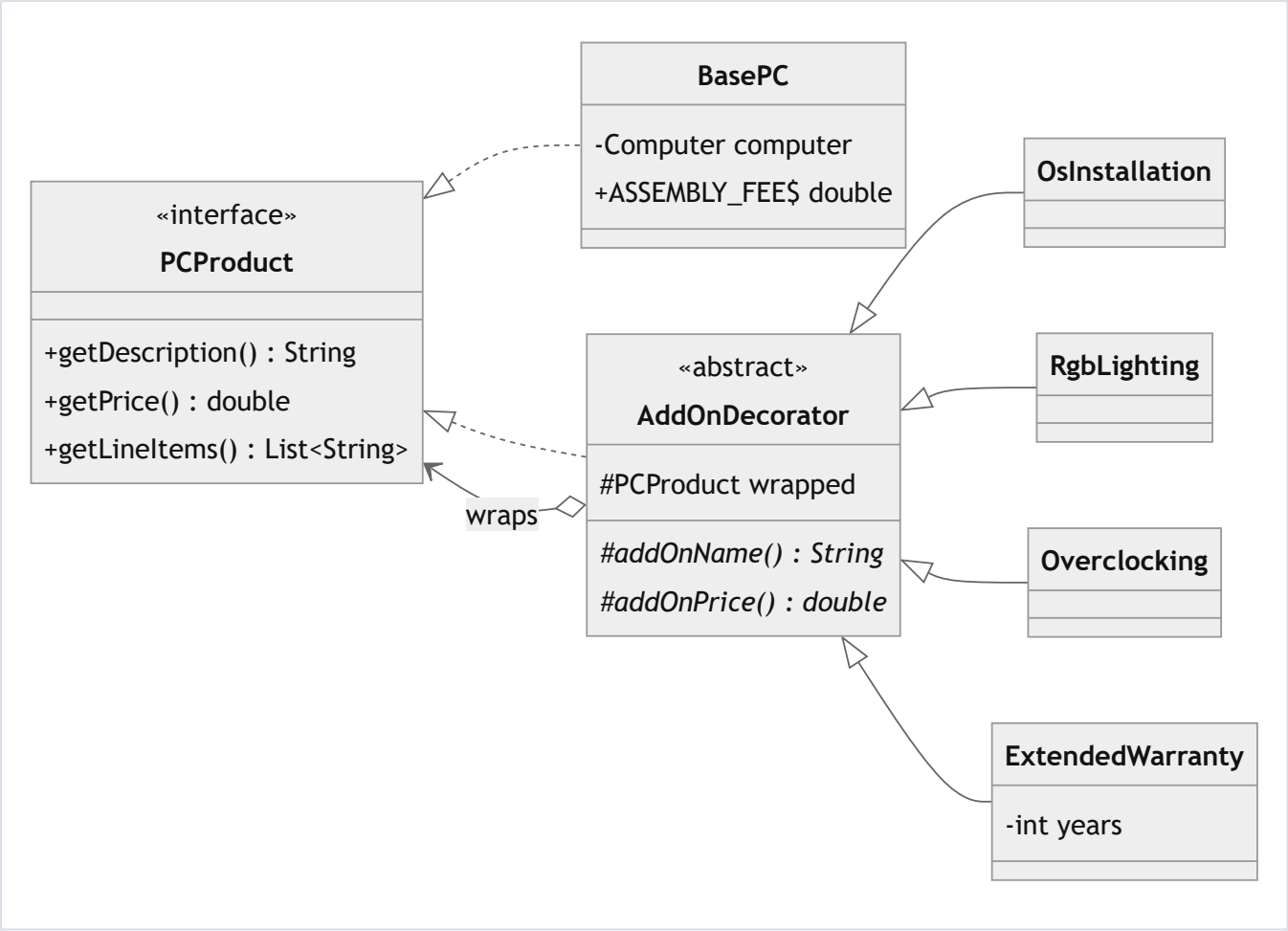
// validate() hands the builder to the chain of compatibility rules (Section 3.11)
public List<String> validate() {
    return CompatibilityChecker.check(this, rules);   // rules = CompatibilityRules.standardChain()
}
```

3.5 Decorator — service add-ons Structural

Problem. On top of the hardware the customer may order any combination of four services. Creating a subclass for every combination ("PC with Windows and RGB but no warranty") would mean $2^4 = 16$ classes, and services must be chosen at runtime.

Solution. `PCProduct` is the common interface. `BasePC` (the concrete component) prices the hardware plus assembly. `AddOnDecorator` holds another `PCProduct` and adds its own name and price; concrete

decorators can be stacked in any order. `ExtendedWarranty` shows a real benefit of decoration: its price is 6 % per year *of everything it wraps*, including the other add-ons.




```
public abstract class AddOnDecorator implements PCProduct {
    protected final PCProduct wrapped;
    protected AddOnDecorator(PCProduct wrapped) { this.wrapped = wrapped; }
    protected abstract String addOnName();
    protected abstract double addOnPrice();

    @Override public String getDescription() { return wrapped.getDescription() + " + " + addOnName(); }
    @Override public double getPrice()       { return wrapped.getPrice() + addOnPrice(); }
}

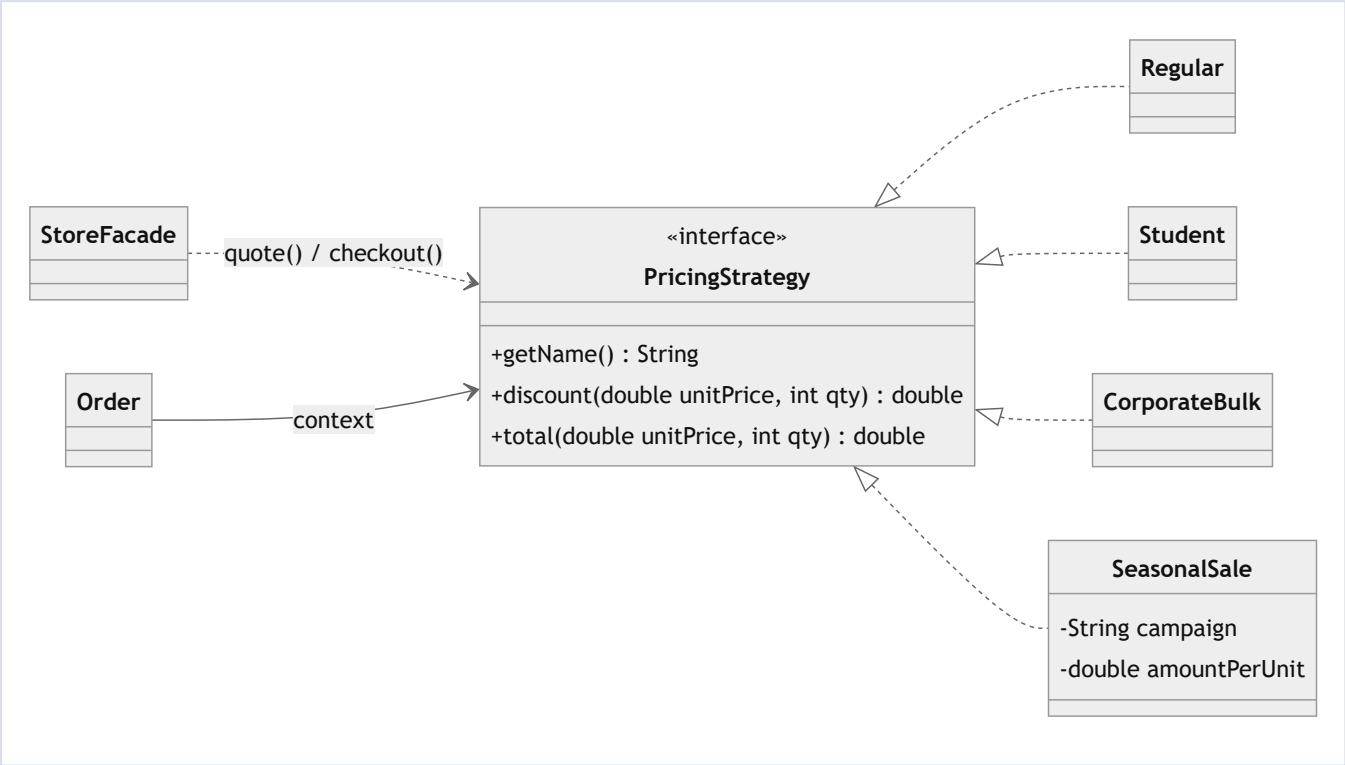
public static class ExtendedWarranty extends AddOnDecorator {
    private final int years;
    @Override protected double addOnPrice() {           // depends on what it wraps
        return Math.round(wrapped.getPrice() * 0.06 * years * 100) / 100.0;
    }
}

// Facade composes the chain at runtime from the ticked check boxes
PCProduct product = new BasePC(computer);
if (addOns.contains(AddOnOption.OS_INSTALL)) product = new AddOns.OsInstallation(product);
if (addOns.contains(AddOnOption.RGB))        product = new AddOns.RgbLighting(product);
if (addOns.contains(AddOnOption.WARRANTY))   product = new AddOns.ExtendedWarranty(product, 2);
```

3.6 Strategy — pricing policies Behavioral

Problem. The same PC is sold at different prices: full price, 10 % student discount capped at $150, volume tiers for companies (5+ machines −7 %, 10+ −12 %), and campaign sales such as "Black Friday, −$120 per PC". Hard-coding these rules in the checkout would produce a growing `switch` and break the Open/Closed principle.

Solution. Every policy implements `PricingStrategy`. The quote, the checkout and the `Order` (the contexts) just call `pricing.total(unitPrice, qty)`. The user swaps strategies at runtime in a combo box, and a new campaign is a new class (or a new `SeasonalSale` instance) without touching existing code.




```
public interface PricingStrategy {
    String getName();
    double discount(double unitPrice, int quantity);
    default double total(double unitPrice, int quantity) {
        double subtotal = unitPrice * quantity;
        return Math.round((subtotal - discount(unitPrice, quantity)) * 100) / 100.0;
    }
}

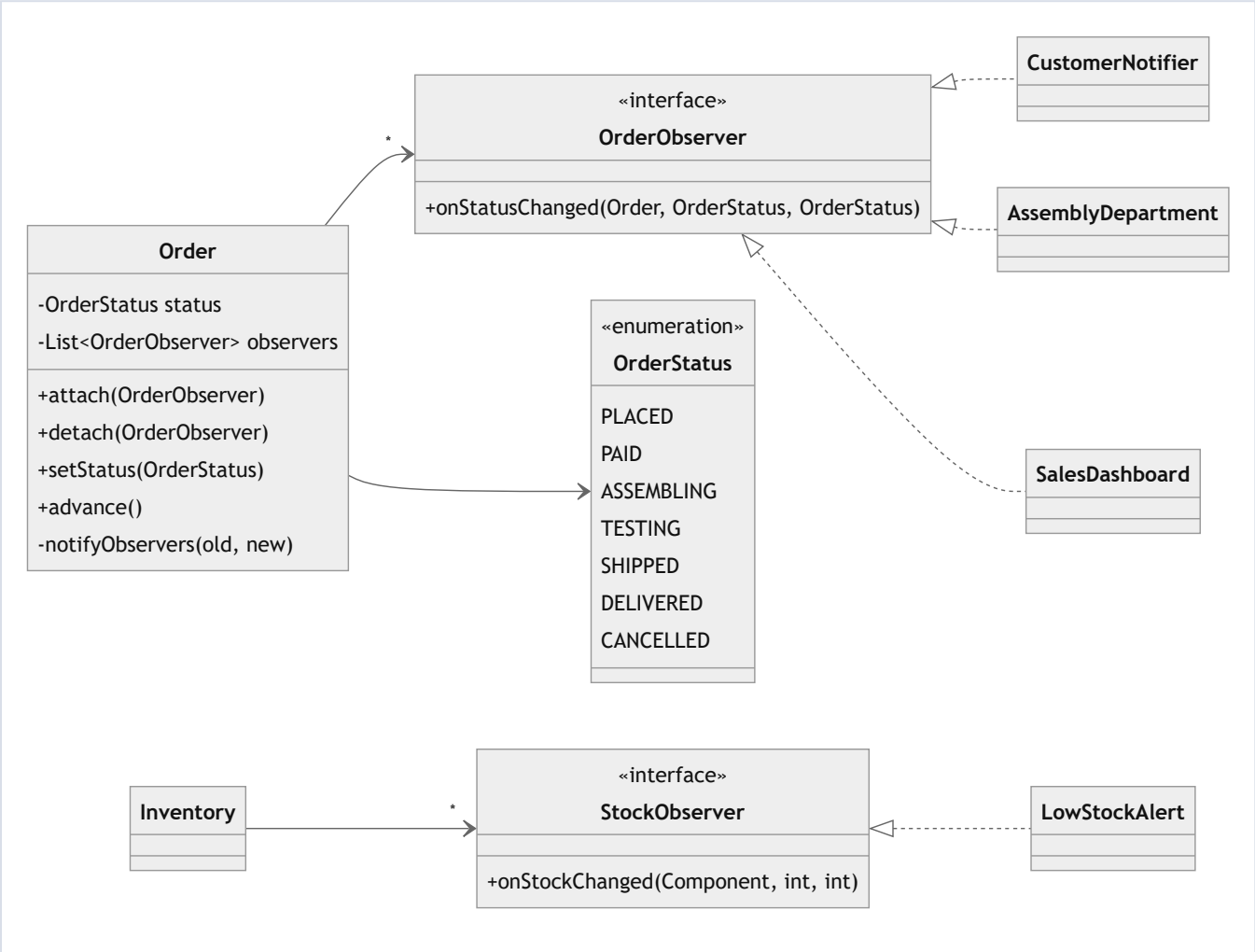
public static class CorporateBulk implements PricingStrategy {
    public double discount(double unitPrice, int quantity) {
        double rate = quantity >= 10 ? 0.12 : quantity >= 5 ? 0.07 : 0.0;
        return unitPrice * quantity * rate;
    }
}

public static class Student implements PricingStrategy {
    public double discount(double unitPrice, int quantity) {
        return Math.min(unitPrice * quantity * 0.10, 150.0);
    }
}
```

3.7 Observer — order and stock events Behavioral

Problem. When an order changes status, several independent parties must react: the customer receives an e-mail, the workshop queues the build and runs a stress test, and the sales dashboard updates revenue. When stock runs low, purchasing must be warned. The order and the inventory should not know about any of these departments.

Solution. The pattern is used twice. `Order` is a subject for `OrderObserver`s (`CustomerNotifier`, `AssemblyDepartment`, `SalesDashboard`); the `Inventory` singleton is a subject for `StockObserver`s (`LowStockAlert`). Subjects keep a list of observers and push every change. Observers write to a log sink, which the GUI shows in the "Event log" panel. Adding an SMS notifier means writing one class and attaching it — no change to `Order`.



```
public class Order {
    private final List<OrderObserver> observers = new CopyOnWriteArrayList<>();
    public void attach(OrderObserver o) { observers.add(o); }

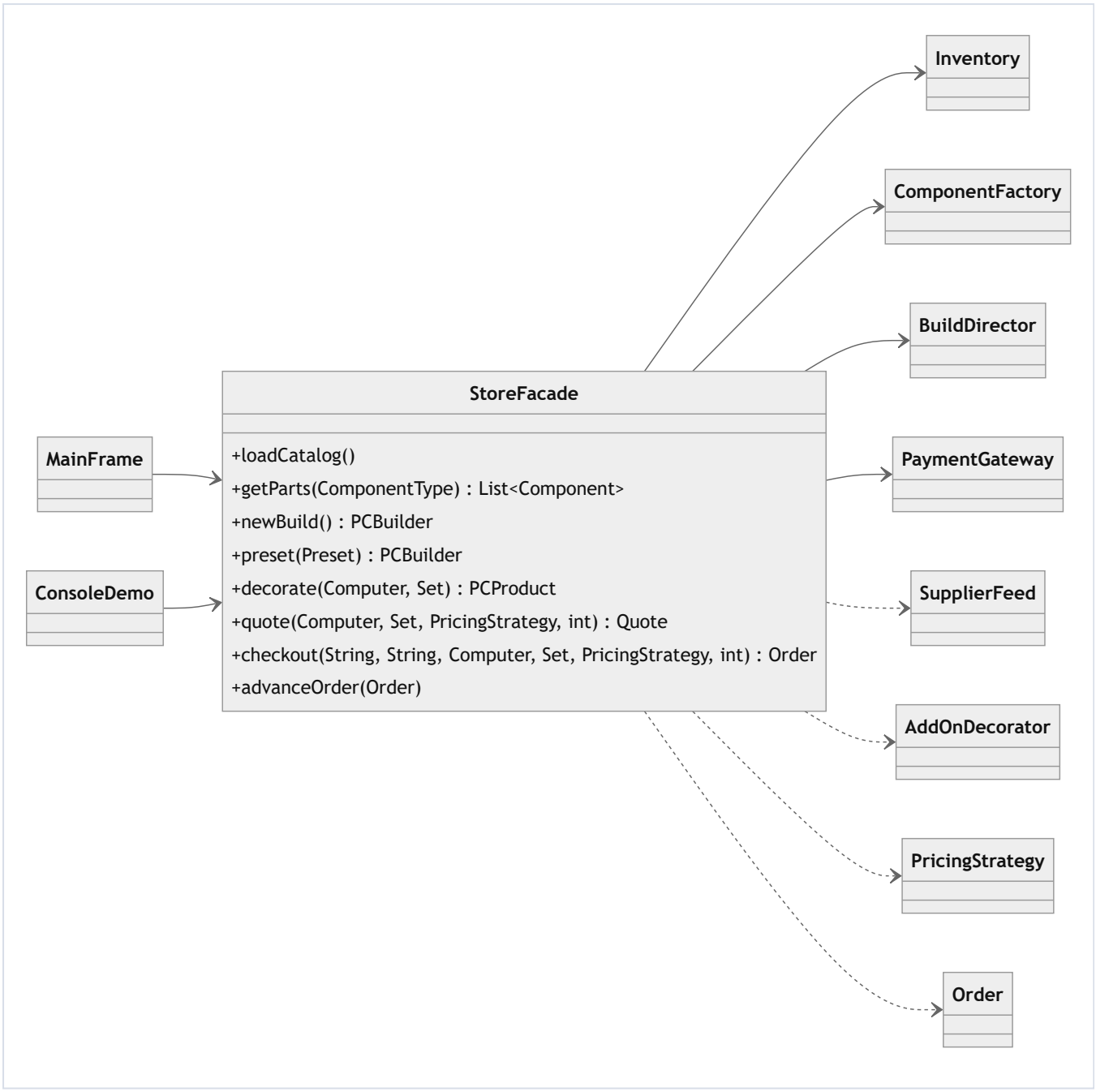
    public void setStatus(OrderStatus newStatus) {
        OrderStatus old = this.status;
        if (old == newStatus) return;
        this.status = newStatus;
        for (OrderObserver o : observers) o.onStatusChanged(this, old, newStatus);
    }
}

public static class AssemblyDepartment implements OrderObserver {
    public void onStatusChanged(Order order, OrderStatus oldStatus, OrderStatus newStatus) {
        if (newStatus == OrderStatus.PAID)
            log.accept("[Workshop] Queued " + order.getQuantity() + " x '"
                    + order.getProduct().getDescription() + "' for assembly");
        else if (newStatus == OrderStatus.TESTING)
            log.accept("[Workshop] Running 4h stress test on " + order.getId());
    }
}
```

3.8 Facade — `StoreFacade` Structural

Problem. Placing one order involves many steps across many subsystems: decorate the PC with add-ons, price it with the right strategy, authorize the card, reserve every part in the inventory, capture the payment (or release the card hold if stock is short), create the order, attach its observers and mark it paid. Putting this logic into Swing event handlers would couple the UI to every class in the system.

Solution. `StoreFacade` exposes a small API — `loadCatalog()`, `getParts()`, `preset()`, `quote()`, `checkout()`, `advanceOrder()` — and hides the subsystems behind it. Both the Swing window and the console demo use only this class. The subsystems remain accessible on their own (e.g. unit tests can use `PCBuilder` directly), which is the usual property of a facade.


Inventory
ComponentFactory
BuildDirector
StoreFacade
+loadCatalog()
+getParts(ComponentType) : List<Component>
+newBuild() : PCBuilder
+preset(Preset) : PCBuilder
+decorate(Computer, Set) : PCProduct
+quote(Computer, Set, PricingStrategy, int) : Quote
+checkout(String, String, Computer, Set, PricingStrategy, int) : Order
+advanceOrder(Order)
MainFrame
ConsoleDemo
PaymentGateway
SupplierFeed
AddOnDecorator
PricingStrategy
Order

```
public Order checkout(String customer, String cardLast4, Computer computer,
                      Set<AddOnOption> addOns, PricingStrategy pricing, int quantity) {
    Quote q = quote(computer, addOns, pricing, quantity);         // Decorator + Strategy

    String auth = payments.authorize(customer, cardLast4, q.getTotal()); // declined? stop here

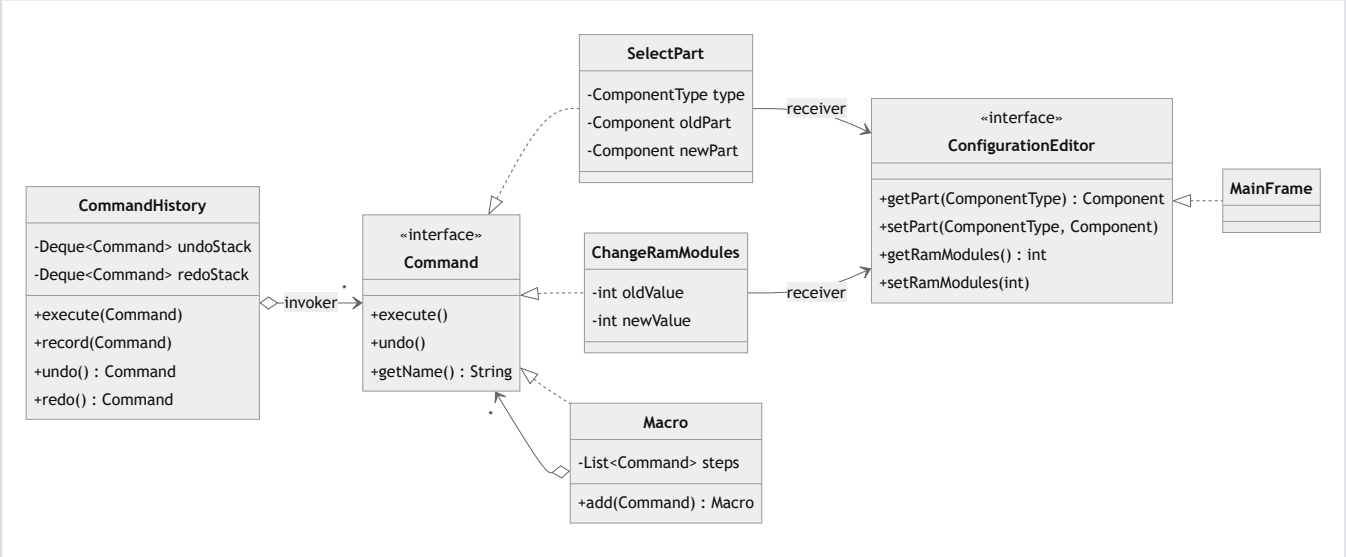
    List<Component> units = new ArrayList<>();
    for (int i = 0; i < quantity; i++) units.addAll(computer.allParts());
    try {
        inventory.reserve(units);                                  // Singleton, all-or-nothing
    } catch (RuntimeException e) {
        payments.voidAuthorization(auth);                          // release the card hold
        throw e;
    }
    String tx = payments.capture(auth);

    Order order = new Order(customer, q.getProduct(), quantity, pricing);
    orderObservers.forEach(order::attach);                         // Observer
    orders.add(order);
    order.setStatus(OrderStatus.PAID);                             // notifies everyone
    return order;
}
```

3.9 Command — undo / redo Behavioral

Problem. While configuring a PC the customer experiments: tries a bigger graphics card, sees that the power supply is now too weak, changes the case, and then wants to go back. Without history every mistake must be repaired by hand. Loading a preset changes up to eight parts at once, and undoing it part by part would be tedious.

Solution. Every change is an object implementing `Command` with `execute()` and `undo()`. `SelectPart` stores the category, the old and the new part; `ChangeRamModules` stores the old and new count; `Macro` groups several commands (a preset load) so they are undone as *one* step. `CommandHistory` is the invoker with an undo stack and a redo stack (a new command clears redo, history is limited to 100 steps). The receiver is the `ConfigurationEditor` interface; the Swing window implements it, and the tests use a simple in-memory implementation, so the commands are independent of the GUI. In the application the buttons *Undo*/*Redo* and the shortcuts Ctrl+Z / Ctrl+Y use this history.



```
public static class SelectPart implements Command {
    private final ConfigurationEditor editor;
    private final ComponentType type;
    private final Component oldPart, newPart;

    @Override public void execute() { editor.setPart(type, newPart); }
    @Override public void undo()    { editor.setPart(type, oldPart); }
}

public static class Macro implements Command {            // one undo step for a whole preset
    private final List<Command> steps = new ArrayList<>();
    @Override public void execute() { for (Command c : steps) c.execute(); }
    @Override public void undo() {
        for (int i = steps.size() - 1; i >= 0; i--) steps.get(i).undo();   // reverse order
    }
}

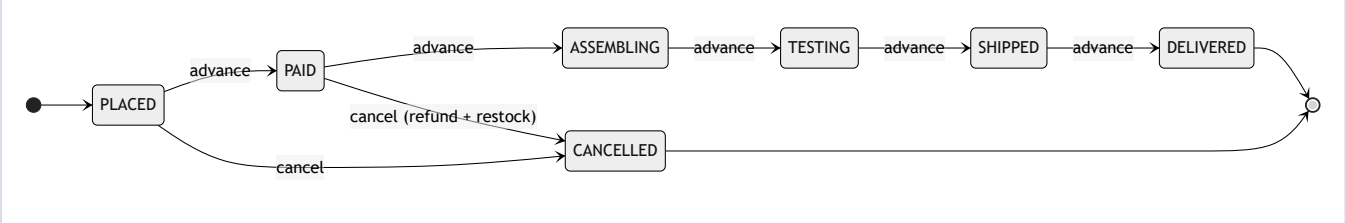
public class CommandHistory {                                // the invoker
    private final Deque<Command> undoStack = new ArrayDeque<>();
    private final Deque<Command> redoStack = new ArrayDeque<>();

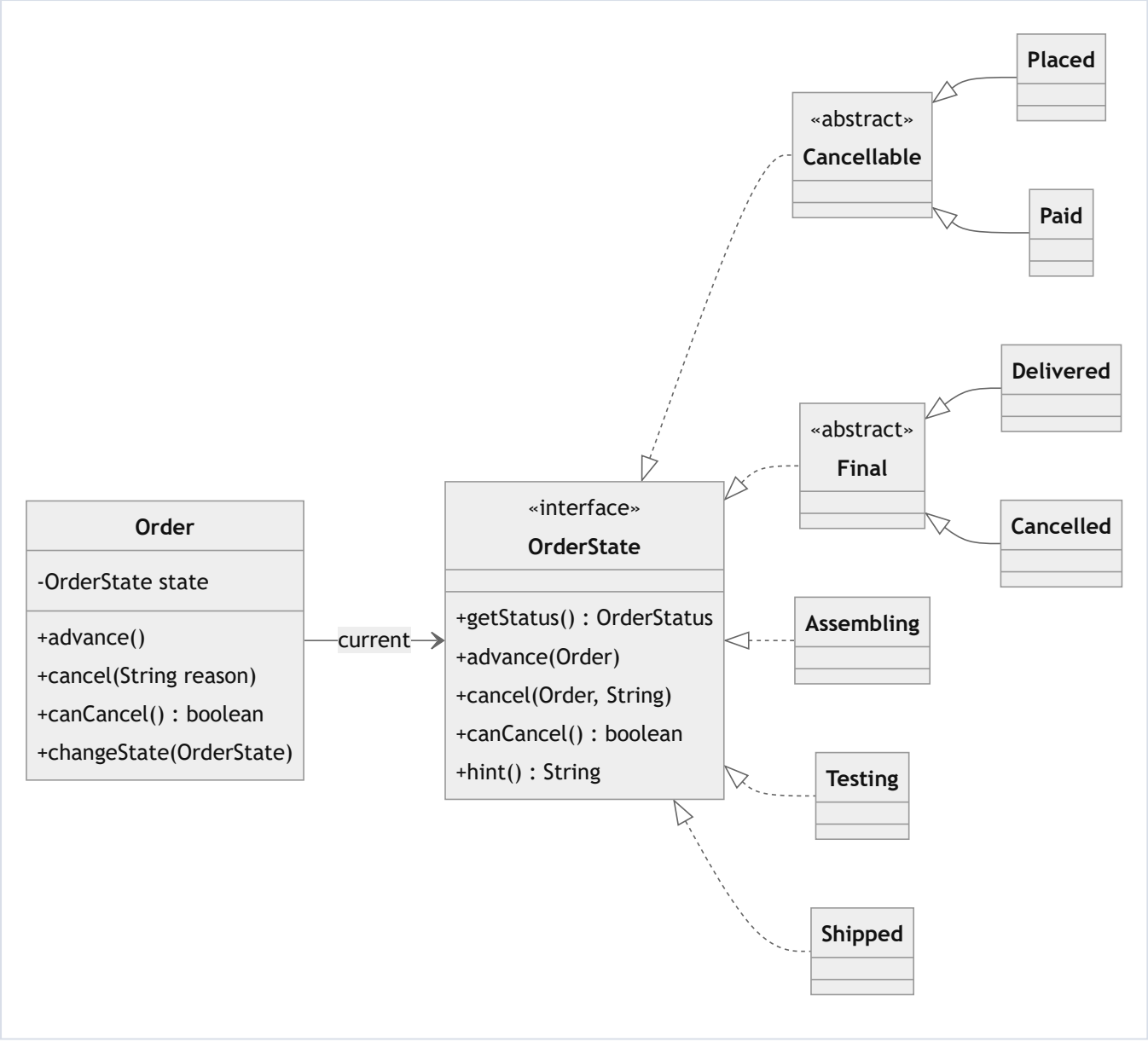
    public void execute(Command c) { c.execute(); undoStack.push(c); redoStack.clear(); }
    public Command undo() {
        if (undoStack.isEmpty()) return null;
        Command c = undoStack.pop(); c.undo(); redoStack.push(c); return c;
    }
    public Command redo() {
        if (redoStack.isEmpty()) return null;
        Command c = redoStack.pop(); c.execute(); undoStack.push(c); return c;
    }
}
```

3.10 State — order life cycle Behavioral

Problem. What an order may do depends on its step. A paid order waiting in the queue may be cancelled with a full refund; an order on the workbench may not; a delivered or cancelled order must not move any further. Writing this as `if (status == PAID || status == PLACED)` in every method of `Order`, the facade and the GUI spreads the rules everywhere and breaks as soon as a new step (e.g. "returned") is added.

Solution. Each step is a class implementing `OrderState` with `advance()`, `cancel()`, `canCancel()` and `hint()`. `Order` is the *context*: it keeps a reference to its current state and delegates to it. States decide the next step themselves (`Paid.advance()` switches to `Assembling`), and `Order.changeState()` notifies the observers, so State and Observer work together. Common behaviour sits in small abstract bases: `Cancellable` (PLACED, PAID) and `Final` (DELIVERED, CANCELLED, which reject any further step). States hold no data, so each is a shared constant. The GUI simply asks `order.canCancel()` to enable or disable the "Cancel" button.






```
public class Order {                                  // the context
    private OrderState state = OrderStates.PLACED;
    public void advance()               { state.advance(this); }
    public void cancel(String reason)   { state.cancel(this, reason); }
    public boolean canCancel()          { return state.canCancel(); }

    public void changeState(OrderState next) {         // called by the states
        OrderStatus old = state.getStatus();
        state = next;
        for (OrderObserver o : observers) o.onStatusChanged(this, old, next.getStatus());
    }
}

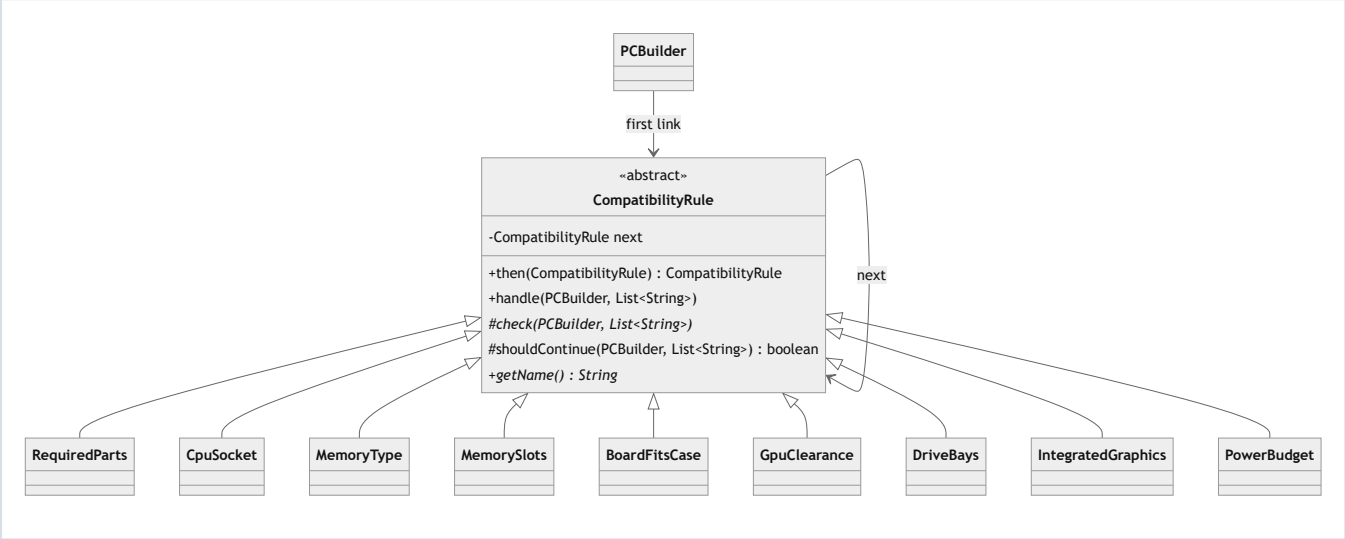
static final class Paid extends Cancellable {
    public OrderStatus getStatus()     { return OrderStatus.PAID; }
    public void advance(Order order)   { order.changeState(ASSEMBLING); }
}

static final class Assembling extends Base {          // Base.cancel() throws
    public void advance(Order order)   { order.changeState(TESTING); }
    protected String whyNoCancel()     { return "the PC is already being assembled"; }
}
```

3.11 Chain of Responsibility — compatibility rules Behavioral

Problem. The first version checked compatibility in one long method with a dozen `if` blocks. Every new rule (for example, mini-ITX boards have only two RAM slots, small cases have fewer drive bays) made that method longer, and a shop could not add its own rule (such as a budget limit for a school order) without editing it.

Solution. Every rule is a handler extending `CompatibilityRule`. A handler checks one concern, records problems and passes the request to the next handler. `handle()` is a template that runs `check()` and then forwards; a rule may override `shouldContinue()` to stop the chain.
`CompatibilityRules.standardChain()` links nine rules, and the builder only knows the first link. A different chain can be given to a builder with `rules(chain)`; the tests append a custom budget rule this way without changing any existing class. By default all rules run so the customer sees every problem at once.



```
public abstract class CompatibilityRule {
    private CompatibilityRule next;
    public CompatibilityRule then(CompatibilityRule next) { this.next = next; return next; }

    public final void handle(PCBuilder build, List<String> issues) {
        check(build, issues);
        if (next != null && shouldContinue(build, issues)) next.handle(build, issues);
    }
    protected abstract void check(PCBuilder build, List<String> issues);
    protected boolean shouldContinue(PCBuilder build, List<String> issues) { return true; }
}

public static CompatibilityRule standardChain() {
    CompatibilityRule first = new RequiredParts();
    first.then(new CpuSocket()).then(new MemoryType()).then(new MemorySlots())
         .then(new BoardFitsCase()).then(new GpuClearance()).then(new DriveBays())
         .then(new IntegratedGraphics()).then(new PowerBudget());
    return first;
}

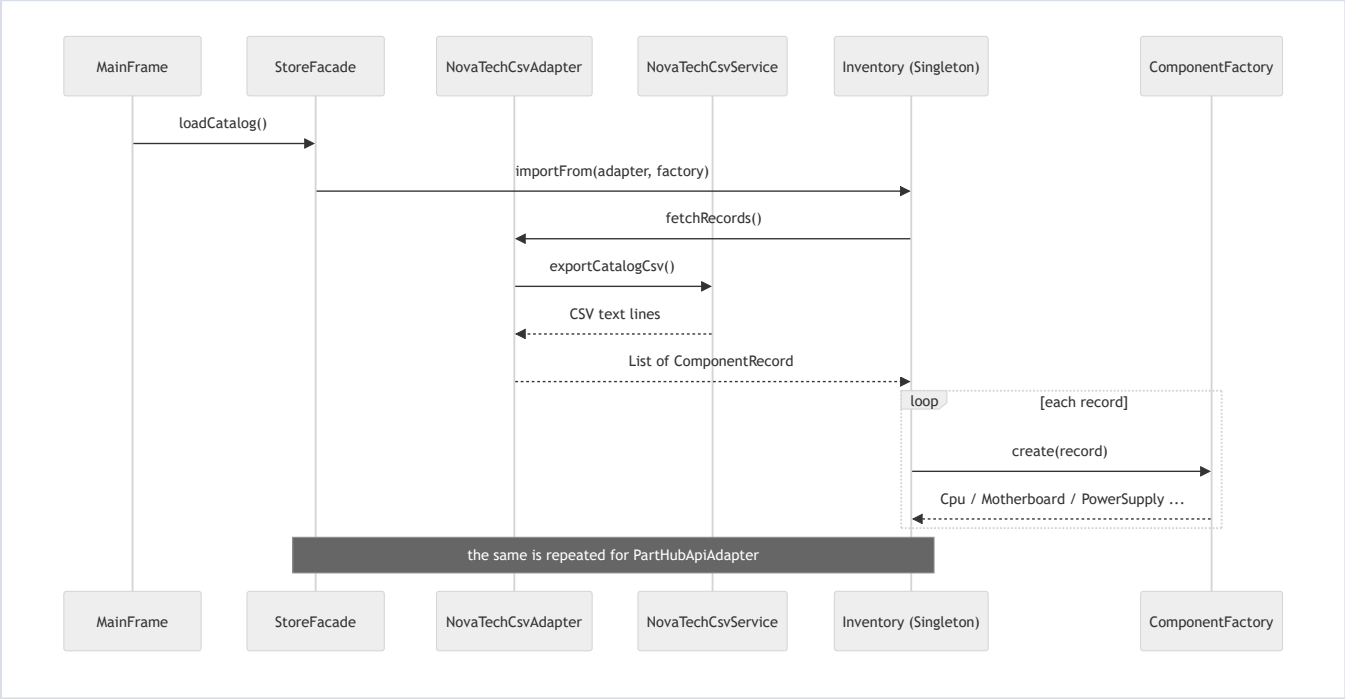
public static class GpuClearance extends CompatibilityRule {
    protected void check(PCBuilder b, List<String> issues) {
        if (b.getGpu() != null && b.getCase() != null
                && b.getGpu().getLengthMm() > b.getCase().getMaxGpuLengthMm())
            issues.add("Graphics card is " + b.getGpu().getLengthMm() + " mm but the case fits only "
                    + b.getCase().getMaxGpuLengthMm() + " mm.");
    }
}
```

4. How the patterns cooperate

The patterns are not isolated examples; they form one pipeline. The sequence diagrams below show the two main scenarios.

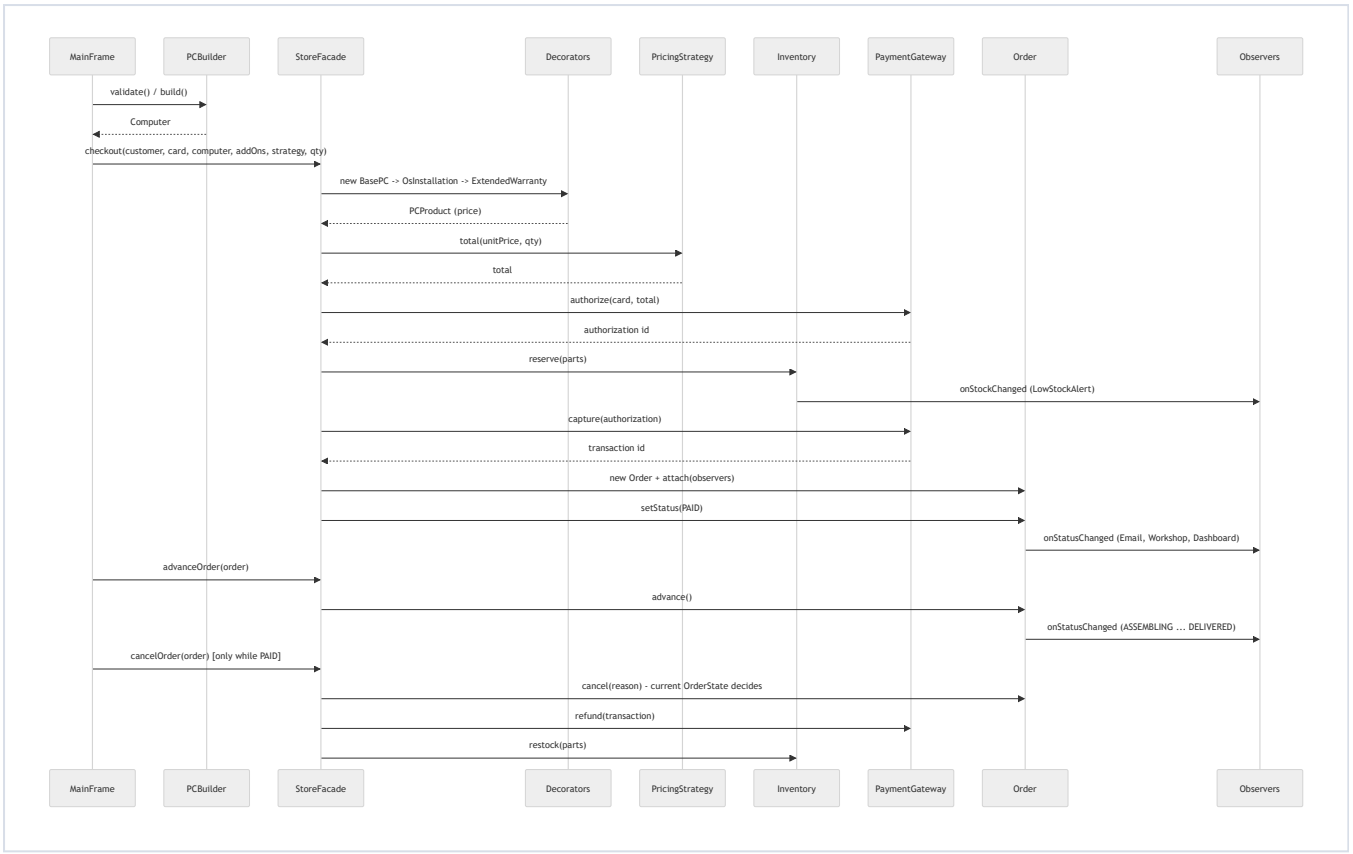
4.1 Start-up: loading the catalog

Adapter → Factory → Singleton: the facade asks each supplier adapter for records, the factory turns each record into a concrete part, and the parts are stored in the single inventory.



4.2 Checkout and order tracking

Builder + Chain of Responsibility → Facade → Decorator → Strategy → Singleton → Observer → State.


MainFrame
PCBuilder
StoreFacade
Decorators
PricingStrategy
Inventory
PaymentGateway
Order
Observers
validate() / build()
Computer
checkout(customer, card, computer, addOns, strategy, qty)
new BasePC -> OsInstallation -> ExtendedWarranty
PCProduct (price)
total(unitPrice, qty)
total
authorize(card, total)
authorization id
reserve(parts)
onStockChanged (LowStockAlert)
capture(authorization)
transaction id
new Order + attach(observers)
setStatus(PAID)
onStatusChanged (Email, Workshop, Dashboard)
advanceOrder(order)
advance()
onStatusChanged (ASSEMBLING ... DELIVERED)
cancelOrder(order) [only while PAID]
cancel(reason) - current OrderState decides
refund(transaction)
restock(parts)
MainFrame
PCBuilder
StoreFacade
Decorators
PricingStrategy
Inventory
PaymentGateway
Order
Observers

5. Application screenshots

The screenshots were produced by the application itself (`run.bat --screenshots docs\screenshots` drives the real GUI through a scripted scenario and saves PNG files).

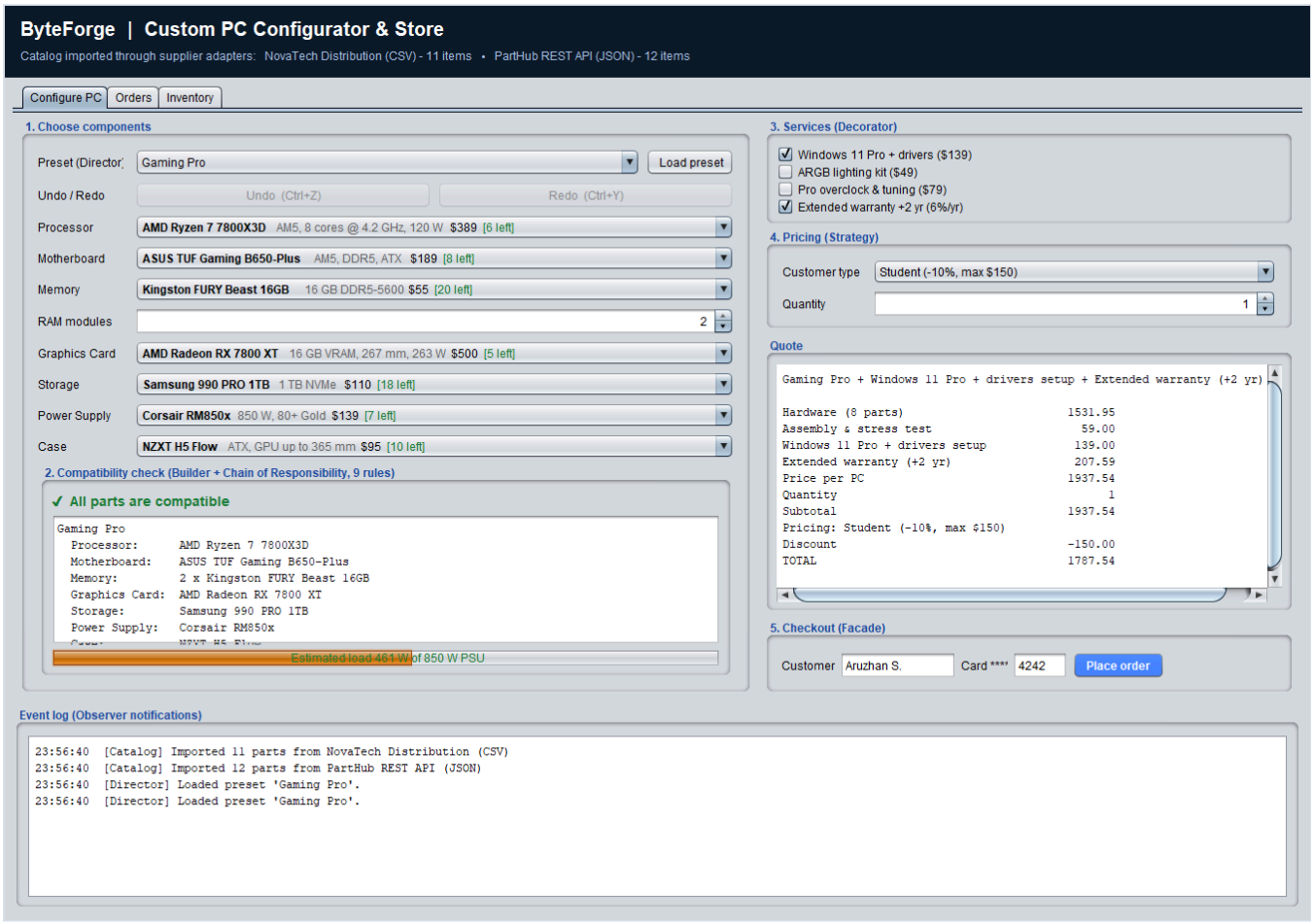


Figure 5.1 – "Gaming Pro" preset loaded by the **Director**, validated by the **Builder**, with Windows and warranty add-ons (**Decorator**) and student pricing (**Strategy**).

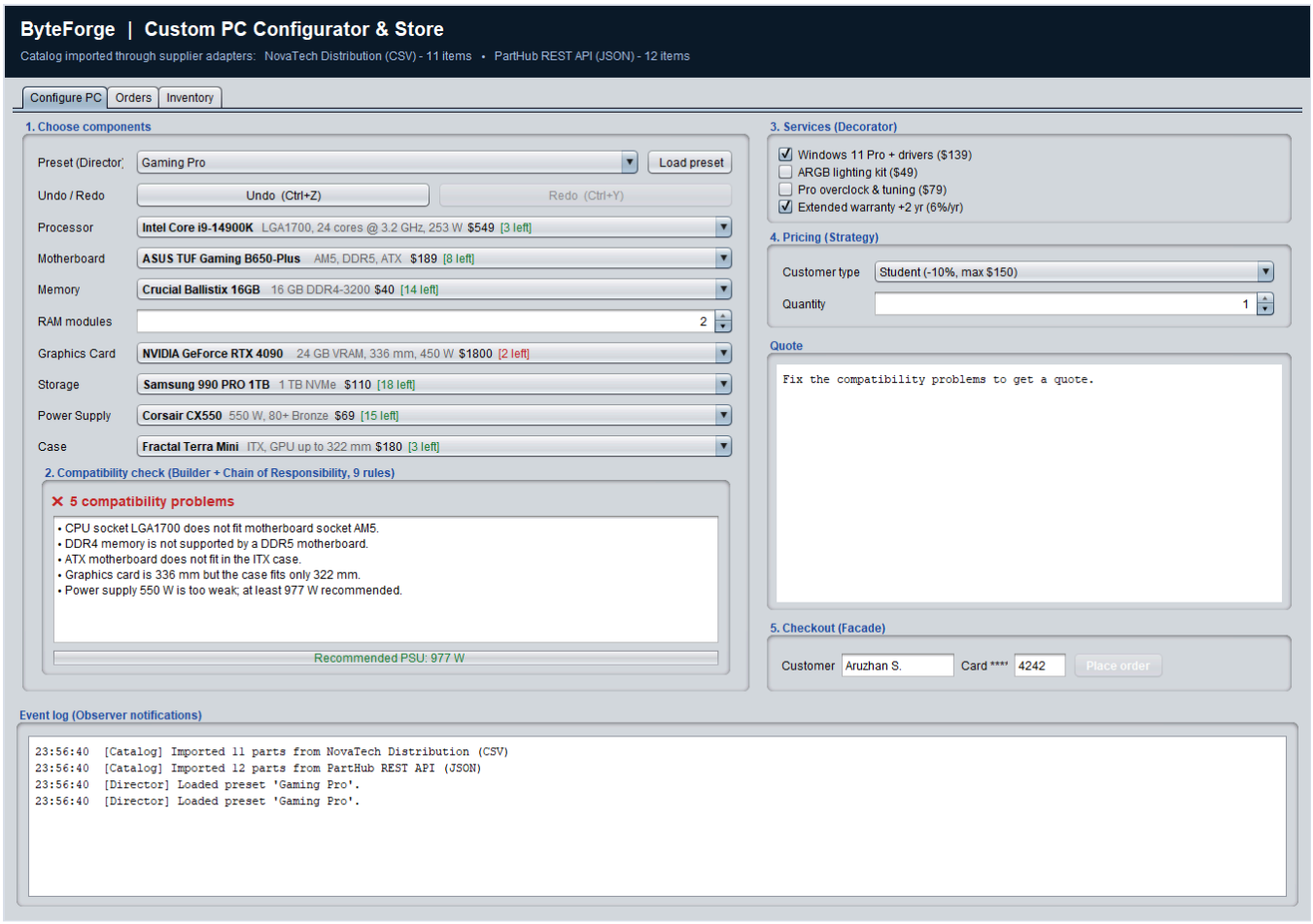


Figure 5.2 – An incompatible configuration: the **Builder** lists five problems (socket, DDR type, form factor, GPU length, PSU wattage) and checkout is disabled.

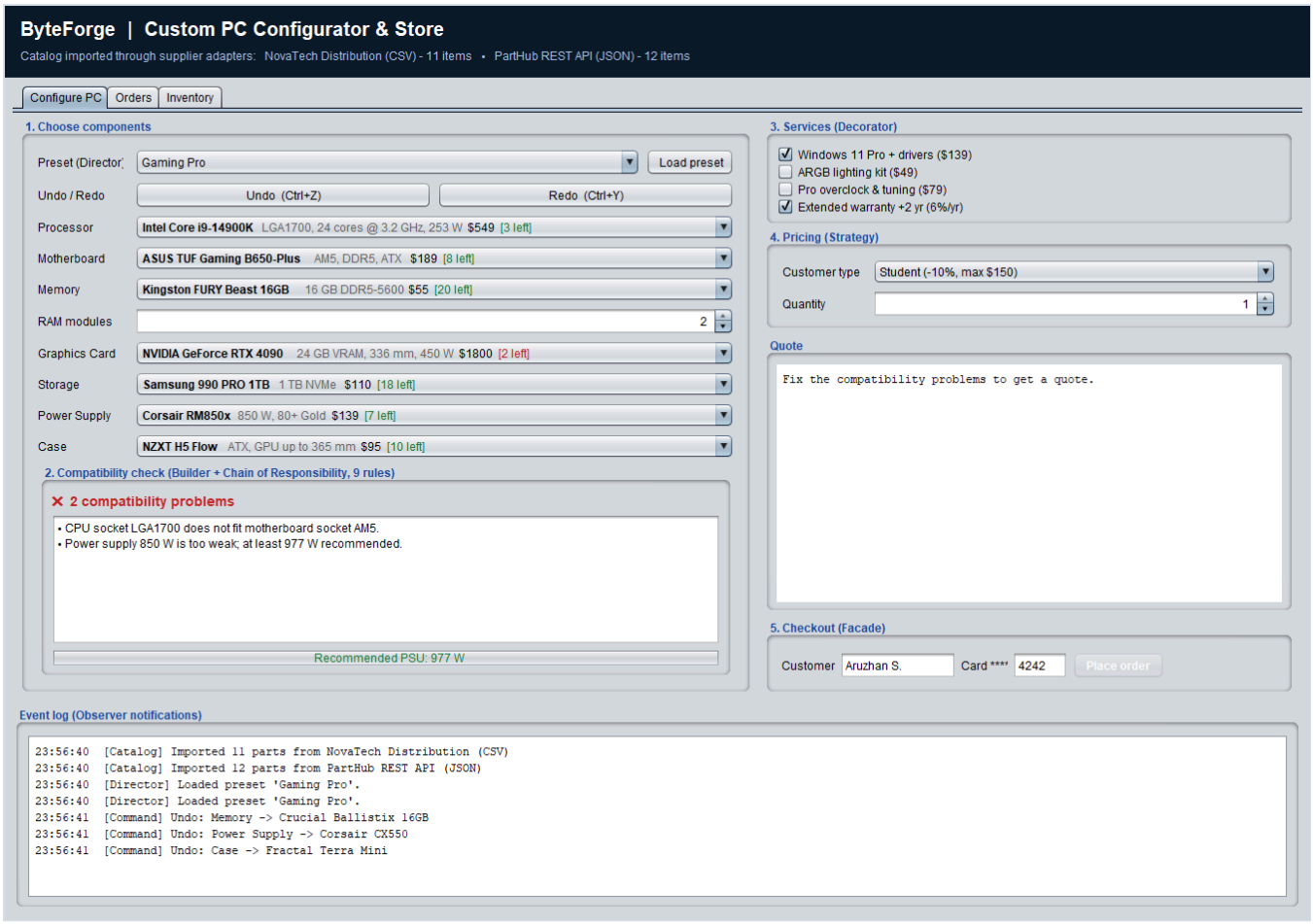


Figure 5.3 – Three changes undone with **Command** history (memory, power supply and case are restored; see the event log). Five problems are reduced to two.

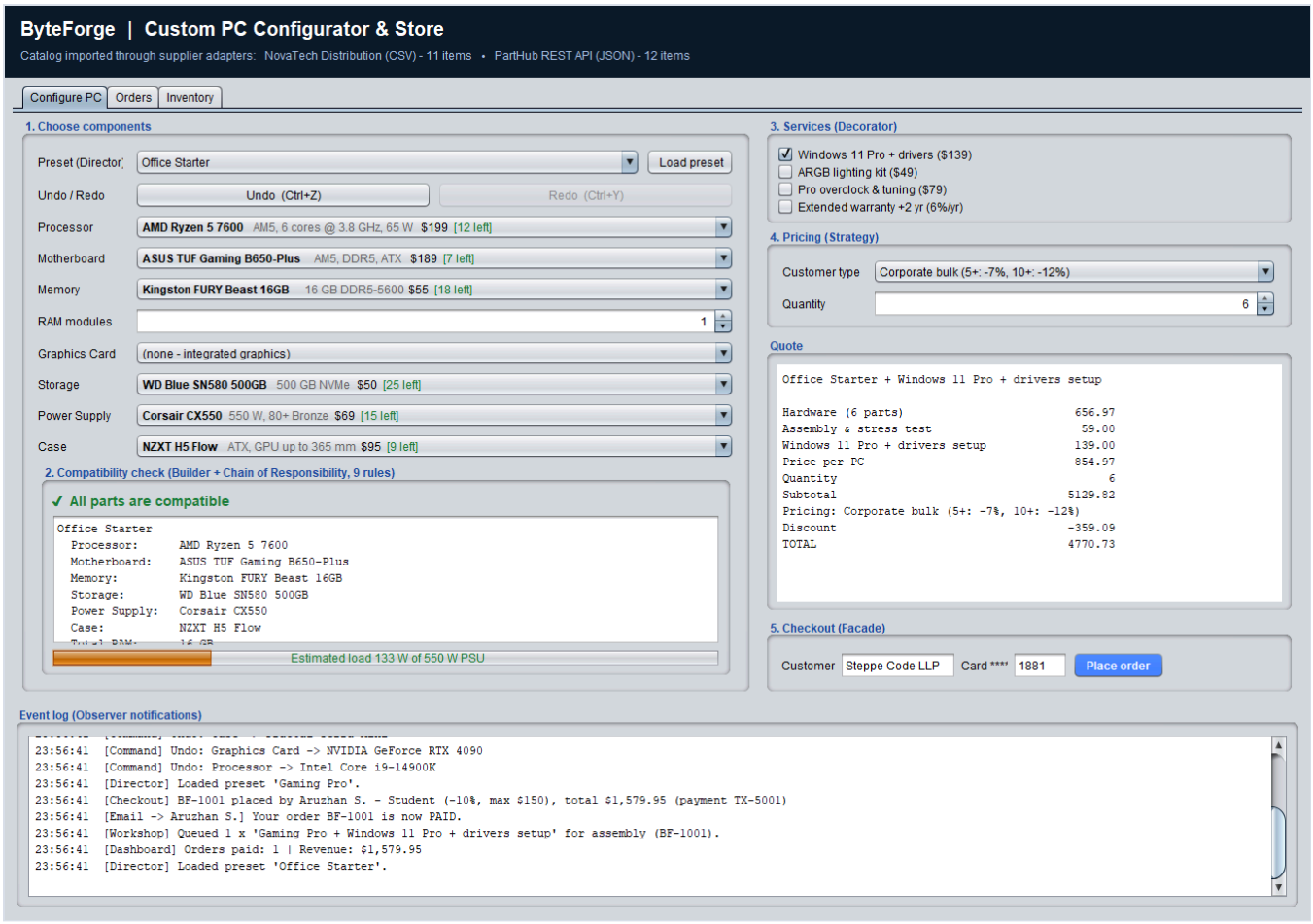


Figure 5.4 – Six "Office Starter" PCs for a company: the corporate bulk **Strategy** applies a 7 % discount. The event log shows **Observer** notifications from the first order.

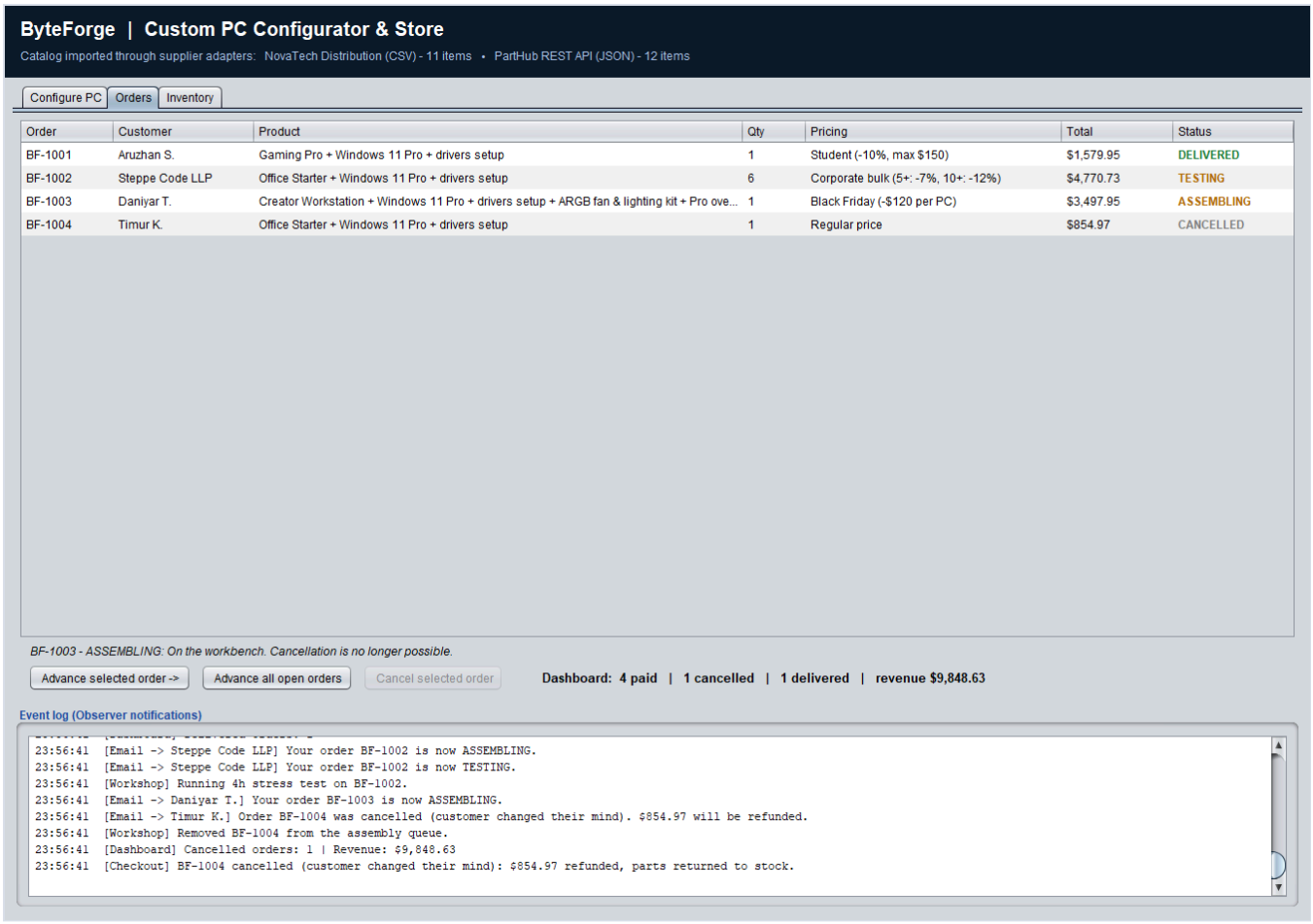


Figure 5.5 – Orders tab: four orders in different **states**. BF-1004 was cancelled while PAID (refund + parts back to stock); for the selected BF-1003 (ASSEMBLING) the state disables the Cancel button. Every change is pushed to the customer, workshop and dashboard (**Observer**).

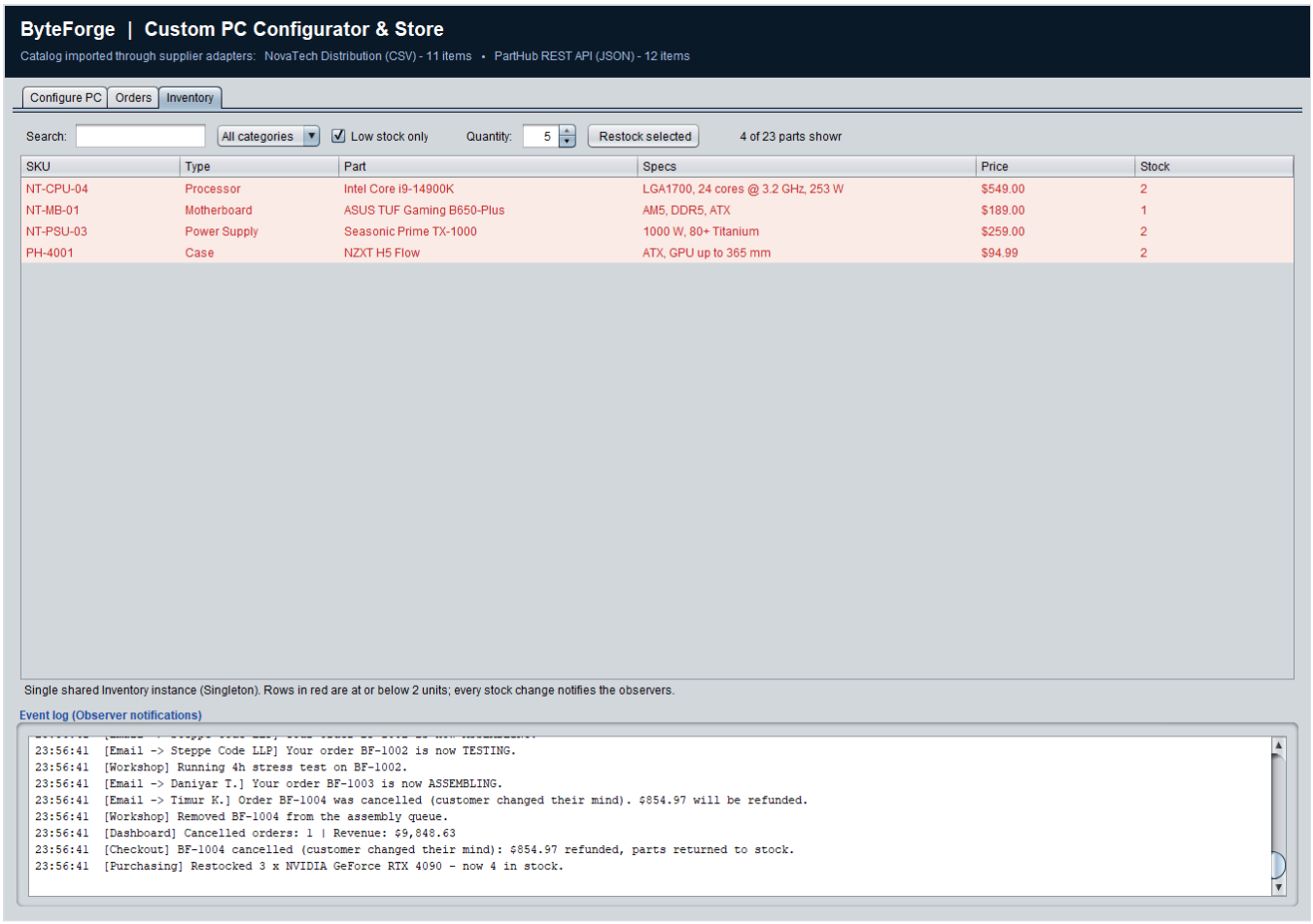


Figure 5.6 – Inventory tab of the single shared stock (**Singleton**) imported through **Adapter** + **Factory**, filtered to "low stock only". Three RTX 4090 were just restocked, so the card left the list; the event log shows the delivery.

6. Testing and results

6.1 Automated tests

The project contains its own small test runner (`test/com/byteforge/PatternTests.java`) so that the tests run on any machine with a JDK, without downloading JUnit. Every test starts from a freshly loaded catalog. Command: `run.bat --test` (exit code 1 if any test fails). Result of the last run: **45 passed, 0 failed**.

Pattern	Test	Result
Singleton	getInstance() always returns the same object	**Pass**
Singleton	constructor is private	**Pass**
Singleton	reserve() is all-or-nothing	**Pass**
Factory	creates the right subclass for each type	**Pass**
Factory	missing attribute is reported clearly	**Pass**
Adapter	CSV supplier: 11 records with parsed attributes	**Pass**
Adapter	JSON supplier: categories mapped, cents converted	**Pass**
Adapter	catalog contains 23 parts from both suppliers	**Pass**
Builder	all three presets are valid	**Pass**
Builder	socket mismatch is rejected	**Pass**
Builder	DDR4 on a DDR5 board is rejected	**Pass**
Builder	GPU longer than the case is rejected	**Pass**
Builder	weak PSU is rejected with a recommendation	**Pass**
Builder	F-series CPU without GPU is rejected	**Pass**
Builder	incomplete build lists every missing part	**Pass**
Builder	built Computer is immutable	**Pass**
Chain	standard chain contains 9 rules in a fixed order	**Pass**
Chain	every rule runs, so all problems are reported at once	**Pass**
Chain	ITX board accepts at most 2 RAM modules	**Pass**
Chain	ITX case has only 2 drive bays	**Pass**
Chain	a new rule can be appended without changing existing ones	**Pass**
Chain	a rule can stop the chain	**Pass**
Decorator	base price = hardware + assembly	**Pass**

Decorator	add-ons stack and describe themselves	**Pass**
Decorator	warranty price includes what it wraps	**Pass**
Strategy	regular price has no discount	**Pass**
Strategy	student discount is 10% capped at $150	**Pass**
Strategy	corporate tiers: 4 = 0%, 5 = 7%, 10 = 12%	**Pass**
Strategy	seasonal sale is per unit and never negative	**Pass**
Observer	all attached observers receive each change	**Pass**
State	life cycle PLACED -> ... -> DELIVERED, then no further step	**Pass**
State	only PLACED and PAID orders can be cancelled	**Pass**
State	cancelling while ASSEMBLING is rejected with a reason	**Pass**
State	cancelled order cannot advance	**Pass**
Facade	cancelling a paid order refunds and restocks	**Pass**
Observer	low-stock alert fires at the threshold	**Pass**
Facade	checkout reserves stock, charges and notifies	**Pass**
Facade	declined card leaves stock untouched	**Pass**
Facade	out of stock releases the card hold	**Pass**
Facade	restock adds units and rejects non-positive quantities	**Pass**
Facade	quote equals the order total	**Pass**
Command	undo restores the previous part, redo re-applies	**Pass**
Command	macro command is undone as one step	**Pass**
Command	a new command clears the redo stack	**Pass**
Command	history is limited	**Pass**

6.2 Console demonstration

The console mode (`run.bat --console`) runs a fixed scenario through the patterns. Shortened output:

```
=== 2. BUILDER + CHAIN OF RESPONSIBILITY: a rejected custom build =====
   Rule chain: Required parts -> CPU socket -> Memory type -> Memory slots -> Board fits case
               -> GPU clearance -> Drive bays -> Integrated graphics -> Power budget
   x CPU socket LGA1700 does not fit motherboard socket AM5.
   x DDR4 memory is not supported by a DDR5 motherboard.
   x ATX motherboard does not fit in the ITX case.
   x Graphics card is 336 mm but the case fits only 322 mm.
   x Power supply 550 W is too weak; at least 845 W recommended.

=== 3. DECORATOR + STRATEGY: same PC, different add-ons and pricing ====
   Gaming Pro + Windows 11 Pro + drivers setup + Extended warranty (+2 yr)
   Hardware (8 parts)                       1531.95
   Assembly & stress test                     59.00
   Windows 11 Pro + drivers setup            139.00
   Extended warranty (+2 yr)                 207.59
   TOTAL                                    1937.54

   Regular price                           qty 1  -> $  1,729.95
   Student (-10%, max $150)                qty 1  -> $  1,579.95
   Corporate bulk (5+: -7%, 10+: -12%)     qty 6  -> $  9,653.12
   Black Friday (-$120 per PC)             qty 1  -> $  1,609.95

=== 4. FACADE + OBSERVER: checkout and order tracking ==================
   [Checkout] BF-1001 placed by Aruzhan S. - Student (-10%, max $150), total $1,579.95
   [Email -> Aruzhan S.] Your order BF-1001 is now PAID.
   [Workshop] Queued 1 x 'Gaming Pro + Windows 11 Pro + drivers setup' for assembly (BF-1001).
   [Dashboard] Orders paid: 1 | Revenue: $1,579.95
   ...
   [Email -> Aruzhan S.] Your order BF-1001 is now DELIVERED.

=== 6. FACADE error handling: a declined card never touches stock ======
   RX 7800 XT stock before: 4
   [Checkout] Payment declined for Bad Card Inc.: Card ****0000 was declined
   RX 7800 XT stock after:  4

=== 7. STATE: cancelling depends on where the order is ==================
   BF-1003 is PAID -> Paid, waiting in the workshop queue. Can still be cancelled with a full refund.
   [Checkout] BF-1003 cancelled (customer changed their mind): $715.97 refunded, parts returned to
stock.
   BF-1004 is ASSEMBLING -> On the workbench. Cancellation is no longer possible.
   Refused: Order BF-1004 cannot be cancelled: the PC is already being assembled

=== 8. COMMAND: undo / redo of configuration changes ===================
   After 3 commands: GPU = NVIDIA GeForce RTX 4090, RAM modules = 4
   Undo: RAM modules 2 -> 4
   Undo: Graphics Card -> NVIDIA GeForce RTX 4090
   Now:  GPU = NVIDIA GeForce RTX 4060, RAM modules = 2
   Redo: Graphics Card -> NVIDIA GeForce RTX 4090
```

6.3 A defect found by testing

The first version reserved stock *before* charging the card and returned the parts if the card was declined. The test "declined card leaves stock untouched" showed that this still produced *false low-stock alerts*: the observers saw the stock drop and react before the roll-back. We changed the facade to the two-step flow used by real payment gateways (authorize → reserve → capture). Thanks to the Facade pattern this change touched only `StoreFacade` and `PaymentGateway`; the GUI did not change at all.

7. Conclusion

We designed and implemented ByteForge, a custom PC configurator and computer-store application in Java. The project integrates **eleven** design patterns — three creational (Singleton, Factory, Builder), three structural (Adapter, Decorator, Facade) and five behavioral (Strategy, Observer, Command, State, Chain of Responsibility) — into a single working system, far beyond the required six. Every team member implemented three or four patterns, and all patterns are covered by 45 automated tests.

The patterns are not isolated demonstrations: they form one chain. Supplier **adapters** feed a **factory** that fills the **singleton** inventory; the **builder**, using a **chain** of compatibility rules, guarantees that only physically compatible computers can exist, and every change of the configuration is a **command** that can be undone; **decorators** add services; a **strategy** prices the result; the **facade** runs checkout; the order's **state** decides what may happen next; and **observers** keep customers, the workshop, management and purchasing informed.

The main lessons we learned:

- Patterns pay off when requirements change. Adding a new supplier, part type, service, pricing campaign or notification channel requires a new class, not edits throughout the code (Open/Closed principle).
- The Builder's validation turned a domain rule ("only compatible PCs") into a guarantee enforced by the type system: an invalid `Computer` object cannot be created.
- The Facade kept the GUI thin. The same facade serves the Swing window, the console demo and the tests, and the payment-flow fix (Section 6.3) did not require any GUI change.
- Defining the Command receiver as an interface made undo/redo testable without starting the GUI.
- Replacing a long method with a Chain of Responsibility and a status switch with State made each rule and each step a small class with one job, which was easy to test in isolation.
- Patterns also have costs: more classes and indirection. We kept each pattern where it solved a real problem in the domain rather than adding patterns for their own sake.

8. Further work

- **Persistence.** Store the inventory and orders in a database (e.g. PostgreSQL via JDBC) behind a Repository; the Singleton would then wrap a connection pool.
- **Real supplier integration.** Replace the simulated services with real HTTP calls; thanks to the Adapter pattern only the adapter classes change.
- **Returns and warranty claims** as new order states (DELIVERED → RETURN_REQUESTED → REFUNDED), and a rule editor in the GUI that enables or disables chain links.
- **Recommendation strategy:** suggest the cheapest upgrade that fixes a compatibility problem (e.g. "choose a 1000 W PSU").
- **Benchmarks:** estimate gaming FPS or rendering time per configuration using public benchmark data.
- **Build tooling:** migrate the tests to JUnit 5 with a Maven/Gradle build and CI; a web front end (Spring Boot) reusing the same facade.

Appendix A. Project structure and how to run

```
ByteForge/
├── run.bat                     compile + run: GUI, --console, --test, --screenshots <dir>
├── README.md
├── ByteForge.iml, .idea/       IntelliJ IDEA project with 3 run configurations
├── docs/
│   ├── report/Report.pdf       this report (Report.html is its source)
│   ├── screenshots/            PNG screenshots
│   ├── console_output.txt      full console demo output
│   └── test_output.txt         full test run output
├── test/com/byteforge/
│   └── PatternTests.java       45 automated tests
└── src/com/byteforge/
    ├── Main.java               entry point
    ├── ConsoleDemo.java        text demo of all patterns
    ├── model/                  Component, ComponentType, Parts (Cpu, Gpu, Ram, ...)
    ├── adapter/                SupplierFeed, NovaTechCsvAdapter/Service, PartHubApiAdapter/Client
    ├── factory/                ComponentFactory, ComponentRecord
    ├── inventory/              Inventory (Singleton)
    ├── builder/                PCBuilder, Computer, BuildDirector, CompatibilityChecker
    ├── decorator/              PCProduct, BasePC, AddOnDecorator, AddOns
    ├── strategy/               PricingStrategy, PricingStrategies
    ├── observer/               Order, OrderStatus, OrderObserver, StockObserver, Listeners
    ├── facade/                 StoreFacade, PaymentGateway, AddOnOption
    ├── command/                Command, ConfigCommands, CommandHistory, ConfigurationEditor
    ├── state/                  OrderState, OrderStates
    ├── rules/                  CompatibilityRule, CompatibilityRules
    └── ui/                     MainFrame, ScreenshotTour
```

Running

- **IntelliJ IDEA:** File → Open → the `ByteForge` folder. Choose the run configuration *ByteForge GUI*, *ByteForge Console demo* or *ByteForge Tests*. If IntelliJ asks for an SDK, select any JDK 11+.
- **Command line (Windows):** `run.bat` (GUI), `run.bat --console`, `run.bat --test`. Requires JDK 11 or newer.
- **Any OS:** `javac -d out $(find src test -name "*.java")` then `java -cp out com.byteforge.Main`.

Suggested defense scenario (5 minutes)

1. Start the GUI: the header shows two suppliers imported through **adapters**; the Inventory tab shows the **singleton** stock created by the **factory**.
2. Load "Gaming Pro" (**Director**), then choose an Intel CPU and the RTX 4090 → the rule **chain** lists the problems and the **Builder** refuses to build.
3. Press Ctrl+Z twice (**Command**) → the configuration is valid again.
4. Tick Windows + warranty (**Decorator**), switch customer type (**Strategy**) and watch the quote change.
5. Place the order (**Facade**), open the Orders tab and advance it → notifications in the event log (**Observer**).
6. Place a second order and cancel it while PAID; try to cancel the one being assembled → refused (**State**).

7. Try card **0000** → payment declined, stock unchanged. Finish with `run.bat --test`: 45 passed.